

Multiagentní systémy

Učební text ke státní závěrečné zkoušce

SZZ Umělá inteligence, MFF UK

srpen 2026

Text pokrývá okruh „Multiagentní systémy“ magisterské SZZ oboru Umělá inteligence. Kapitoly jsou vedené **jedna ku jedné** podle vět oficiálního znění okruhu: každé větě okruhu odpovídá právě jedna kapitola, jak ukazuje mapovací tabulka níže. Podkladem jsou slidy přednášky NAIL106 *Multiagentní systémy* (Neruda, Pilát, MFF UK), doplněné o učebnice M. Wooldridge: *An Introduction to Multi-Agent Systems* (2. vyd., 2009), Y. Shoham, K. Leyton-Brown: *Multiagent Systems* (2009) a S. Russell, P. Norvig: *Artificial Intelligence: A Modern Approach*. Anglické termíny jsou při prvním výskytu uvedeny kurzívou v závorce, protože literatura k oboru je výhradně anglická a zkoušející je běžně používá.

Rozsah textu a značení

Rozsah jednotlivých kapitol odpovídá váze látky. Nejvíce prostoru dostávají architektury agentů, komunikace a kooperace s teorií her, tedy témata, kterým se přednáška věnuje nejpodrobněji. Značka **♦ nad rámec slidů** upozorňuje na látku, která **není ve slidech** přednášky. V textu přesto zůstává, a to ze dvou důvodů: buď ji *jmenuje oficiální znění okruhu*, nebo by bez ní okolní výklad nedával smysl. Je to tedy nabídka ke škrtnutí: co označeno není, je ve slidech doloženo. Ve slidech chybějí celé kapitoly **3** (hledání cesty) a **6** (učení agentů), protože je přednáška explicitně uvádí mezi nepokrytými tématy. Okruh je ale vyjmenovává, takže je nutné je umět. Jsou proto zpracovány stručně, na úrovni, která na zkoušce obstojí.

Mapování okruhu na kapitoly

Věta oficiálního znění okruhu	Kapitola v tomto textu
Architektura autonomního agenta; mechanismus výběru akcí.	1 Architektura autonomního agenta a mechanismus výběru akcí
Metody pro řízení agentů; symbolické a konekcionistické reaktivní plánování, hybridní přístupy.	2 Metody pro řízení agentů
Problém hledání cesty.	3 Problém hledání cesty
Komunikace a znalosti v multiagentních systémech, ontologie, řečové akty, FIPA-ACL, protokoly.	4 Komunikace a znalosti v MAS
Distribučované řešení problémů, kooperace, Nashova rovnováha, Paretoova efektivita, alokace zdrojů, aukce.	5 Distribuované řešení problémů, kooperace, teorie her a aukce
Metody pro učení agentů; zpětnovazební učení.	6 Učení agentů a zpětnovazební učení
Metodologie návrhu, jazyky a prostředí multiagentních systémů.	7 Metodologie návrhu, jazyky a prostředí MAS

Obsah

Mapování okruhu na kapitoly	1
1 Architektura autonomního agenta a mechanismus výběru akcí	4
1.1 Agent a multiagentní systém	4
1.2 Vlastnosti inteligentního agenta	5
1.3 Intencionální systémy a intencionální postoj	5
1.4 Prostředí a jeho vlastnosti	5

1.5	Abstraktní architektura agenta	6
1.6	Čistě reaktivní agent a agent s vnitřním stavem	7
1.7	Utilita: jak agentovi říct, co má dělat	8
1.8	Predikátová specifikace úlohy	8
1.9	Mechanismus výběru akcí — přehled	9
1.10	Shrnutí ke zkoušce	9
2	Metody pro řízení agentů: symbolické a konekcionistické reaktivní plánování, hybridní přístupy	10
2.1	Klasický přístup UI a deduktivní agent	10
2.2	Praktické uvažování: architektura BDI	12
2.3	Plánování: STRIPS	12
2.4	Implementace BDI agenta	14
2.5	Odhodlání agenta (<i>commitment</i>)	14
2.6	Procedurální odvozovací systém (PRS)	15
2.7	Reaktivní přístup a jeho východiska	16
2.8	Symbolické reaktivní plánování: subsumpční architektura	17
2.9	Konekcionistické reaktivní plánování: agent network architecture	18
2.10	Omezení reaktivních architektur	19
2.11	Hybridní architektury	19
2.12	Komplexní architektury: IDA a globální pracovní prostor	20
2.13	Srovnání architektur	21
2.14	Shrnutí ke zkoušce	21
3	Problém hledání cesty	22
3.1	Formulace a zařazení	22
3.2	A*	23
3.3	Dynamické prostředí: D* Lite	24
3.4	Hledání cest pro více agentů (MAPF)	25
3.5	Shrnutí ke zkoušce	26
4	Komunikace a znalosti v MAS: ontologie, řečové akty, FIPA-ACL, protokoly	26
4.1	Od agenta k multiagentnímu systému	26
4.2	Co je ontologie	26
4.3	Ontologické jazyky	27
4.4	Uvažování o znalostech: epistemická logika	29
4.5	Proč je komunikace agentů jiná	29
4.6	Teorie řečových aktů	29
4.7	KQML a FIPA-ACL	31
4.8	Interakční protokoly	32
4.9	Shrnutí ke zkoušce	33
5	Distribované řešení problémů, kooperace, teorie her a aukce	34
5.1	Koherence a koordinace	34
5.2	Sdílení úloh a sdílení výsledků	34
5.3	Contract Net	35
5.4	Systémy s tabulí (blackboard)	35
5.5	Distribované omezující podmínky (DisCSP a DCOP)	36
5.6	Interakce agentů v prostředí	36
5.7	Sobecký agent	37
5.8	Hra v normální formě	37
5.9	Klasické hry	38
5.10	Sociální volba a volby	39

5.11	Aukce jako mechanismus alokace zdrojů	40
5.12	Čtyři základní typy aukcí	41
5.13	Kombinatorické aukce a VCG	42
5.14	Další typy aukcí a alokace zdrojů	43
5.15	Návrh mechanismů, VCG a vyjednávání	43
5.16	Shrnutí ke zkoušce	44
6	Metody pro učení agentů a zpětnovazební učení	45
6.1	Proč a jak se agenti učí	45
6.2	Markovský rozhodovací proces	46
6.3	Q-learning a SARSA	47
6.4	Metody založené na politice	48
6.5	Multiagentní zpětnovazební učení	49
6.6	Shrnutí ke zkoušce	50
7	Metodologie návrhu, jazyky a prostředí MAS	50
7.1	Agentově orientované softwarové inženýrství	50
7.2	Standard FIPA a architektura platformy	51
7.3	Jazyky a prostředí	51
7.4	Agenti postavení na velkých jazykových modelech	52
7.5	Shrnutí ke zkoušce	53
	Závěrem: jak na zkoušku	53

1 Architektura autonomního agenta a mechanismus výběru akcí

1.1 Agent a multiagentní systém

Jako samostatná oblast informatiky se multiagentní systémy (*multiagent systems*, MAS) zformovaly v 90. letech, a to z docela praktického důvodu. Výpočetní technika se rozšířila všude a začala být trvale propojená a distribuovaná, takže výpočet přestal být osamocenou úlohou a stal se interakcí. Zároveň rostla složitost zadání natolik, že bylo výhodnější úlohu softwaru delegovat, než ji krok po kroku předsat. K tomu se přidal tlak na snadnost použití.

Klasické paradigma „program dostane vstup, spočítá výstup, skončí“ zde přestává stačit: software je do svého prostředí zasazený trvale a to prostředí se mění nezávisle na něm.

Agent — pracovní definice

Russell, Norvig (AIMA): Agent je cokoli, co lze chápat jako entitu *vnímající* své prostředí prostřednictvím senzorů a *působící* na toto prostředí pomocí efektorů.

Pattie Maes: Autonomní agenti jsou výpočetní systémy, které obývají nějaké složité dynamické prostředí, autonomně v něm vnímají a jednají, a tím naplňují množinu cílů či úkolů, pro které byly navrženy.

Wooldridge, Jennings: Agent je počítačový systém zasazený do nějakého prostředí, schopný v tomto prostředí *autonomního jednání* vedoucího ke splnění delegovaných cílů.

Jednotná definice agenta neexistuje. Franklin a Graesser (1996) v článku *Is it an agent, or just a program?* shromáždili desítky vzájemně nekompatibilních definic. Podstatnější než přesné znění je proto to, co všechny sledují: odlišit agenta od běžné procedury či objektu.

Multiagentní systém

Multiagentní systém je systém složený z více agentů, kteří spolu *interagují* — nejčastěji výměnou zpráv po počítačové síti. Agenti *nemusí sdílet společný cíl*; mohou mít vlastní, i protichůdné zájmy. Proto je nutné studovat mechanismy vyjednávání, koordinace a dynamického rozdělování zdrojů.

Autonomie. Autonomie není vlastnost typu ano/ne, ale škála. Člověk je plně autonomní, metoda v Javě není autonomní vůbec a agent leží někde mezi nimi. Měl by autonomně volit *způsob*, jak zadaný problém vyřešit, tedy vybírat si podcíle. Hlavní cíl si ale nevolí: ten je mu delegován.

Vymezení vůči příbuzným oborům. Šíří okruhu vysvětluje to, že MAS lze číst z pěti různých stran.

- **Softwarové inženýrství.** Agent je dalším stupněm abstrakce v řadě strojový kód → strukturované programy → objekty → distribuované objekty (CORBA) → agenti. Posun proti objektům vystihuje klasický bonmot: „objekty to dělají zadarmo, agenti proto, že chtějí“.
- **Distribuované a paralelní systémy.** Desítky let výzkumu tu vyřešily mutex, deadlock i konsensus, hotová řešení se ale přenést nedají. Agenti jsou totiž *autonomní*, koordinační mechanismy nejsou dané dopředu a všechno se musí vyřešit až za běhu.
- **Umělá inteligence.** Tradiční pohled řadí MAS mezi podoblasti UI. Russell a Norvig to obracejí: cílem UI je podle nich návrh inteligentních agentů, tedy $UI \subseteq MAS$. Etzioni situaci vyhrotil poznámkou, že „MAS je 1 % UI a 99 % informatiky“. MAS techniky UI používá (reprezentace znalostí, plánování), zdůrazňuje však *sociální* aspekty, které klasická UI přehlíží.
- **Teorie her a ekonomie.** Odtud pochází formální aparát pro racionální rozhodování více subjektů. Matematická teorie her přitom řeší hlavně otázky *existence*, zatímco MAS zajímají *výpočetní* aspekty: složitost a praktická použitelnost.
- **Sociologie.** Klíčovým pojmem MAS jsou *společnosti agentů*. Platí tu stejná výhrada jako u vztahu UI a lidské inteligence, totiž že inspirace je vítaná, ale ne povinná (letadla nemávají křídly). Opač-

ným směrem je vazba pevnější: sociologie, ekonomie i ekologie používají agenty jako stavební kámen svých simulačních modelů, tedy *agentové modelování* (*agent-based modelling*, ABM).

1.2 Vlastnosti inteligentního agenta

Inteligentního agenta od pouhého programu odlišují tři vlastnosti, které musí mít současně.

- **Reaktivita** (*reactive*) znamená, že agent vnímá prostředí a dokáže na jeho změny reagovat v rozumném, tedy reálném čase.
- **Proaktivita** (*proactive*) znamená, že agent má vlastní cíle a aktivně je naplňuje z vlastní iniciativy, nikoli až jako odpověď na vnější podnět.
- **Sociálnost** (*social ability*) znamená, že agent umí komunikovat s jinými agenty i s lidmi.

Dosáhnout *čistě* reaktivního nebo *čistě* cílově orientovaného chování je snadné. Obtížné je najít **rovnováhu mezi reaktivitou a proaktivitou**: agent nesmí slepě následovat plán, který se mezitím stal nesmyslným, ale ani nesmí cíle přehodnocovat tak často, že nikam nedorazí. Tento kompromis se vrací v celém textu, nejzřetelněji u odhodlání agenta (odst. 2.5) a u hybridních architektur (odst. 2.11).

1.3 Intencionální systémy a intencionální postoj

Mluvíme-li o agentech, přisuzujeme jim *mentální stavy*: „agent věří, že...“, „agent chce...“. Daniel Dennett takový způsob popisu pojmenoval. **Intencionální systém** je podle něj entita, jejíž chování lze predikovat přisouzením vlastností jako přesvědčení (*beliefs*), touhy (*desires*) a racionalita. Intencionální systémy navíc tvoří hierarchii. Systém *prvního řádu* má přesvědčení a touhy, systém *druhého řádu* má přesvědčení a touhy o přesvědčeních a touhách, svých i cizích, a tak dále.

Dennett rozlišuje tři *postoje* (*stances*), z nichž lze chování systému predikovat. *Fyzikální* postoj vystačí s fyzikálními zákony: hodím-li kamenem, nemusím mluvit o jeho touhách, stačí hmotnost, rychlost a gravitace. *Návrhový* postoj (*design stance*) nastupuje tam, kde všechny fyzikální zákony neznám, vím ale, k jakému účelu byl systém navržen; budík umím nastavit, aniž bych rozuměl jeho elektronice. *Intencionální* postoj popisuje systém pomocí přesvědčení, tužeb a racionality.

Shoham uvádí provokativní příklad: „Vypínač v místnosti je velmi kooperativní agent, který je z vlastní vůle ochoten vést elektrický proud, ale činí tak jen tehdy, věří-li, že si to skutečně přejeme.“ Popis je konzistentní a elegantní, přesto většině lidí připadá absurdní, *protože pro vypínač máme jednodušší a stejně přesný popis*. Intencionální postoj se tedy vyplatí tam, kde fyzikální či návrhový popis neexistuje nebo je nepraktický. I se schématem počítače v ruce je těžké odvodit, proč se po kliknutí na ikonu otevře okno.

Z pohledu informatiky je intencionální postoj především abstrakce pro zvládnutí složitosti, a právě to je pro mnoho programátorů hlavní přínos MAS. Dennett sám k tomu dodává filozofickou otázku: intencionalita podle něj neexistuje sama o sobě, vzniká teprve *v procesu interpretace* pozorovatelem.

1.4 Prostředí a jeho vlastnosti

Chování agenta zásadně určuje typ prostředí, ve kterém se nachází. Prostředí se proto třídí podle čtyř dichotomií; následující klasifikace pochází od Russella a Norviga.

Dichotomie	Význam
plně / částečně pozorovatelné (<i>fully / partially observable</i>)	Senzory agenta mu dávají úplný stav prostředí — nebo jen jeho část (skryté informace, šum, omezený dosah).
statické / dynamické (<i>static / dynamic</i>)	Prostředí se mění pouze v důsledku akcí agenta — nebo i jinak (jiní agenti, přírodní děje). Ve statickém prostředí si agent může „v klidu rozmyslet“ další tah.
deterministické / nedeterministické	Každá akce má právě jeden možný výsledek — nebo více (se známým či neznámým rozdělením pravděpodobnosti; pak mluvíme o stochastickém prostředí).
diskrétní / spojitě	Konečný počet stavů a akcí — nebo spojitě veličiny (poloha, rychlost).

Nepříjemné konstatování: většina zajímavých prostředí, tedy reálný svět i internet, padne po každé do té horší varianty. Jsou spojitá, nedeterministická, dynamická a jen částečně pozorovatelná. Právě proto nestačí naprogramovat agenta jako pevnou tabulku pravidel.

Příklad — termostat. Termostat je agent, byť ne příliš inteligentní, zasazený do prostředí, kterým je místnost. Rozeznává dva vjemy, „je zima“ a „teplota v pořádku“, a má dvě akce, zapnout a vypnout. Jeho rozhodovací jednotka se vejde na jeden řádek: zima $\Rightarrow$ zapni, OK $\Rightarrow$ vypni. Prostředí je přesto nedeterministické, protože třeba někdo otevře dveře.

Příklad — softwarový démon. Vezměme unixový `xbiff`. Jeho prostředím je operační systém, vnímá voláním systémových služeb a jeho akcemi jsou softwarové operace, jako spustit poštovního klienta nebo změnit ikonu. Rozhodovací algoritmus zůstává na úrovni termostatu.

1.5 Abstraktní architektura agenta

Dosavadní popis byl neformální; teď pohled na agenta a jeho interakci s prostředím zformalizujeme. Předpokládáme přitom, že prostředí je diskrétní. Není to omezující předpoklad, protože každé spojitě prostředí lze diskretizovat s libovolnou přesností.

Prostředí, agent, běh

Prostředí může být v jednom z konečně mnoha diskrétních stavů, $E = \{e, e', \dots\}$. **Agent** má k dispozici konečnou množinu akcí $A = \{a, a', \dots\}$.

Běh (*run*) agenta v prostředí je posloupnost střídajících se stavů prostředí a akcí:

$$r = e_0 \xrightarrow{a_0} e_1 \xrightarrow{a_1} e_2 \xrightarrow{a_2} \dots \xrightarrow{a_{n-1}} e_n.$$

Označme R množinu všech konečných běhů (nad E a A), $R^A \subseteq R$ ty, které končí akcí, a $R^E \subseteq R$ ty, které končí stavem prostředí.

Přechodová funkce prostředí (*state transformer function*)

$$\tau : R^A \rightarrow 2^E$$

Funkce τ zobrazuje běh končící akcí na množinu stavů, do nichž se prostředí může po této akci dostat. Z této definice plynou dva důsledky:

- **Prostředí má historii.** Následující stav závisí jen na poslední akci, ale na celé historii běhu.
- **Prostředí je nedeterministické.** Výsledkem je celá množina stavů a dopředu nevíme, který z nich nastane.

Je-li $\tau(r) = \emptyset$, běh skončil. Dále předpokládáme, že všechny běhy skončí.

Prostředí je trojice $Env = \langle E, e_0, \tau \rangle$, kde e_0 je počáteční stav.

Agent a systém

Agent je funkce zobrazující běhy končící stavem prostředí na akce:

$$Ag : R^E \rightarrow A.$$

Agent tedy rozhoduje na základě *celé historie* systému a rozhoduje se *deterministicky*. Označme $\mathcal{AG}$ množinu všech agentů.

Systém je dvojice $\langle Ag, Env \rangle$. Posloupnost $(e_0, a_0, e_1, a_1, \dots)$ je *běh agenta Ag v prostředí Env* , právě když

1. e_0 je počáteční stav Env ,
2. $a_0 = Ag(e_0)$,
3. pro každé $u > 0$ platí $e_u \in \tau((e_0, a_0, \dots, a_{u-1}))$ a $a_u = Ag((e_0, a_0, \dots, e_u))$.

Množinu všech běhů agenta Ag v Env značíme $R(Ag, Env)$.

Funkční ekvivalence agentů

Agenti Ag_1 a Ag_2 jsou **funkčně ekvivalentní vzhledem k Env** , právě když $R(Ag_1, Env) = R(Ag_2, Env)$.

Jsou **prostě funkčně ekvivalentní** (*simply functionally equivalent*), pokud jsou funkčně ekvivalentní vzhledem ke *všem* prostředím.

Funkční ekvivalence slouží ke srovnávání *vyjadřovací síly* architektur. Říká totiž, že na vnitřní implementaci nezáleží, pokud vede ke stejným pozorovatelným běhům.

1.6 Čistě reaktivní agent a agent s vnitřním stavem

Obecná definice agenta počítá s celou historií běhu, a to je hodně silný předpoklad. Nabízejí se proto dvě úspornější varianty. U obou se ptáme na totéž: kolik vyjadřovací síly ta úspora stojí.

Čistě reaktivní agent (*purely reactive / tropistic agent*)

$$Ag : E \rightarrow A$$

Reaguje pouze na aktuální stav prostředí, historii ignoruje.

Vztah k obecnému agentovi: ke každému čistě reaktivnímu agentovi existuje funkčně ekvivalentní standardní agent; opačně to *neplatí*.

Termostat: $Ag(e) = \text{off}$ pro $e = \text{„teplota OK“}$, jinak $Ag(e) = \text{on}$.

Agent s vnitřním stavem

Nechť I je množina vnitřních stavů agenta a Per množina jeho vjemů. Agent je určen trojicí funkcí:

$see : E \rightarrow Per$	(vnímání)
$next : I \times Per \rightarrow I$	(aktualizace vnitřního stavu)
$action : I \rightarrow A$	(výběr akce)

Cyklus: agent startuje ve stavu i_0 ; vnímá prostředí ($see(e)$); aktualizuje stav ($i \leftarrow next(i, see(e))$); vybere akci ($a \leftarrow action(i)$); vykoná ji a opakuje.

Věta (bez důkazu): Ke každému agentovi s vnitřním stavem existuje funkčně ekvivalentní standardní agent. Vnitřní stav tedy *nezvyšuje vyjadřovací sílu*, ale je zásadní pro *efektivitu* a *přehlednost* implementace.

Funkce *see* přitom zároveň formalizuje **pozorovatelnost**. Zobrazí-li dva různé stavy $e_1 \neq e_2$ na tentýž vjem, agent je od sebe nedokáže odlišit a prostředí je pro něj jen částečně pozorovatelné. Krajní

případy jsou dva: buď platí $|Per| = |E|$ a funkce *see* je prostá, což znamená plnou pozorovatelnost, nebo je $|Per| = 1$ a agent nevidí vůbec nic.

1.7 Utilita: jak agentovi říct, co má dělat

Agenty s natvrdo zadrátovaným chováním nechceme. Chceme agentovi říct, *co* má dělat, nikoli *jak*. Standardní postup UI je definovat cíl *nepřímě*, totiž mírou úspěšnosti.

Utilita

Utilita stavu: $u : E \rightarrow \mathbb{R}$. Cílem agenta je dostat se do stavů s vysokou hodnotou utility. Hodnocení jednotlivých stavů je krátkozraké. Proto zavádíme **utilitu běhu**:

$$u : R \rightarrow \mathbb{R}.$$

Ta lépe odpovídá agentům, kteří běží dlouhodobě. Utilitu běhu lze odvodit ze stavů např. jako utilitu *nejhoršího* navštíveného stavu nebo jako *průměrnou* utilitu navštívených stavů; co je lepší, závisí na úloze.

Optimální agent

Nechť $P(r \mid Ag, Env)$ je pravděpodobnost běhu r agenta Ag v prostředí Env , přičemž $\sum_{r \in R(Ag, Env)} P(r \mid Ag, Env) = 1$. Optimální agent maximalizuje *očekávanou utilitu*:

$$Ag_{opt} = \arg \max_{Ag \in AG} \sum_{r \in R(Ag, Env)} u(r) P(r \mid Ag, Env).$$

Definice je elegantní, má ale dvě slabiny. Především je **nekonstruktivní**: neříká, jak takového agenta navrhnout. Kromě toho bývá obtížné utilitní funkci vůbec zadat. Poznamenejme, že peníze nejsou pro člověka dobrou utilitou, protože jejich utilita je nelineární a liší se pro bohaté a chudé.

Příklad: Tileworld. Tileworld je klasické testovací prostředí pro agentní architektury. Agenti se pohybují po šachovnici do čtyř směrů, na desce jsou díry, překážky a bloky a cílem je zatlačit bloky do děr. Prostor je *dynamické*, protože díry, překážky i bloky náhodně vznikají a zanikají. Utilita běhu se měří jako

$$u(r) = \frac{N_f}{N_a},$$

kde N_f je počet děr zaplněných agentem během běhu r a N_a počet všech děr, které se během r objevily. Agent tu musí zvládnout obojí: *reagovat* na změny, když mu zmizí blok, který právě tlačí, a zároveň *využívat příležitosti*, když se vedle něj objeví blok nový. Tileworld se historicky používal právě ke zkoumání kompromisu mezi reaktivitou a proaktivitou (viz odst. 2.5).

1.8 Predikátová specifikace úlohy

Utilita nemusí být jemně odstupňovaná. Speciálním případem je zobrazení do $\{0, 1\}$, kdy se hodnocení scvrkne na jedině rozhodnutí: běh je buď úspěšný ($u(r) = 1$), nebo ne. Tím se popis úlohy zjednoduší na logickou podmínku.

Prostředí úlohy (*task environment*)

Nechť $\Psi : R \rightarrow \{0, 1\}$ je **predikátová specifikace**: $\Psi(r)$ platí, právě když $u(r) = 1$. **Prostředí úlohy** je dvojice $\langle Env, \Psi \rangle$; specifikuje jak vlastnosti systému (přes Env), tak kritérium splnění úlohy (přes Ψ).

Označme $R_\Psi(Ag, Env) = \{r \in R(Ag, Env) \mid \Psi(r)\}$.

Zbývá dohodnout se, kdy vlastně agent úlohu *řeší*. Existují tři výklady a liší se přísností.

- **Pesimistický výklad** žádá, aby podmínku splnily *všechny* běhy, tedy $R_{\Psi}(Ag, Env) = R(Ag, Env)$. Agent úlohu řeší jen tehdy, když nemůže selhat ani v jednom běhu.
- **Optimistický výklad** se spokojí s tím, že Ψ splní *alespoň jeden* běh: $\exists r \in R(Ag, Env) : \Psi(r)$.
- **Realistický výklad** rozšíří τ o rozdělení pravděpodobnosti a měří úspěch pravděpodobností $P(\Psi \mid Ag, Env) = \sum_{r \in R_{\Psi}(Ag, Env)} P(r \mid Ag, Env)$.

Typy úloh

Úlohy dosažení (*achievement tasks*) — „něco najít“. Je dána množina cílových stavů $G \subseteq E$; $\Psi(r)$ platí, pokud se v běhu r vyskytne alespoň jeden stav z G . Typicky každá klasická úloha UI (prohledávání).

Úlohy udržení (*maintenance tasks*) — „něčemu se vyhnout“. Je dána množina „špatných“ stavů $B \subseteq E$; $\Psi(r)$ neplatí, pokud se v r vyskytne stav z B . Typicky hry, kde B jsou prohrané pozice a prostředím je soupeř.

Kombinace: dosáhnout stavu z G a přitom se vyhnout stavům z B .

1.9 Mechanismus výběru akcí — přehled

Mechanismus výběru akcí (*action selection mechanism*)

Procedura, kterou agent v každém okamžiku rozhoduje, kterou z dostupných akcí provede, na základě vjemů z prostředí a svého vnitřního stavu. **Architektura agenta** je softwarová architektura, která takový mechanismus realizuje — podle Maes (1991) „konkrétní metodologie stavby agentů, která určuje, jak lze agenta rozložit na množinu komponent a jak mají tyto komponenty interagovat, aby dohromady odpověděly na otázku, jak data ze senzorů a aktuální vnitřní stav určují akce a budoucí vnitřní stav agenta.“

Výběr akce je **klíčový problém návrhu agenta**. Celá kapitola 2 proto není nic jiného než přehled jeho realizací a následující tabulka shrnuje, jak se dělí do rodin.

Rodina	Mechanismus výběru akce	Typický zástupce
symbolická / delibérativní	logická dedukce nad databází formulí; plánování	deduktivní agent, STRIPS, BDI, PRS
reaktivní (symbolická)	sada pravidel „situace $\rightarrow$ akce“ s pevnou prioritou	subsumpční architektura (Brooks)
reaktivní (konekcionistická)	šíření aktivace v síti modulů, vyhrává nejaktivnější	agent network architecture (Maes)
hybridní	vrstvy různé abstrakce + arbitráž mezi nimi	TouringMachines, InteRRaP
koaliční	soutěž dynamicky vznikajících koalic procesů o „vědomí“	IDA / globální pracovní prostor
naučená	maximalizace naučené Q -funkce / stochastická politika	Q -learning, actor-critic (kap. 6)
generativní	stav se zakóduje do promptu, výstup jazykového modelu se čte jako akce	LLM agenti (§7.4)

1.10 Shrnutí ke zkoušce

- Agent je autonomní systém zasazený do prostředí, který v něm vnímá a jedná, aby splnil delegované cíle. Multiagentní systém je systém interagujících agentů, kteří *nemusí sdílet společný cíl*.
- Inteligentního agenta charakterizují tři vlastnosti: reaktivita, proaktivita a sociálnost. Těžké přitom není dosáhnout jedné z nich, ale vyvážit je.
- Dennett zavádí intencionální systém a tři postoje k predikci chování: fyzikální, návrhový a intencionální. Intencionální postoj je pro informatiku abstrakce pro zvládnutí složitosti.

- Prostředí se třídí podle čtyř dichotomií: pozorovatelnost, statické nebo dynamické, determinismus a diskretnost. Reálná prostředí spadají vždy do nejtěžší varianty.
- Formalismus tvoří stavy E , akce A , běh $r = e_0 a_0 e_1 \dots e_n$, jeho podmnožiny R^A a R^E , přechodová funkce $\tau : R^A \rightarrow 2^E$, z níž plyne historie i nedeterminismus, prostředí $Env = \langle E, e_0, \tau \rangle$, agent $Ag : R^E \rightarrow A$ a systém $\langle Ag, Env \rangle$.
- Funkční ekvivalence se definuje vzhledem k danému prostředí, nebo jako prostá. Čistě reaktivní agent $Ag : E \rightarrow A$ je slabší než obecný agent, kdežto agent s vnitřním stavem (funkce *see*, *next*, *action*) slabší není; oba se dají převést na standardního agenta.
- Utilita se zavádí buď nad stavy jako $u : E \rightarrow \mathbb{R}$, nebo nad běhy jako $u : R \rightarrow \mathbb{R}$. Optimální agent maximalizuje očekávanou utilitu, jenže tato definice je nekonstruktivní. Školním příkladem je Tileworld s utilitou $u(r) = N_f/N_a$.
- Prostředí úlohy je dvojice $\langle Env, \Psi \rangle$; kdy agent úlohu řeší, na to odpovídá pesimista, optimista a realista. Úlohy se dělí na úlohy dosažení (množina G) a úlohy udržení (množina B).
- **Mechanismus výběru akcí** je jádrem celé otázky. U zkoušky se očekává, že student vyjmenuje rodiny mechanismů i jejich typické zástupce podle tabulky výše.

2 Metody pro řízení agentů: symbolické a konekcionistické reaktivní plánování, hybridní přístupy

Tato kapitola je jádrem celého okruhu. Probírá jednotlivé *metody řízení agenta*, tedy konkrétní realizace mechanismu výběru akcí, který zavádí odst. 1.9. Výklad postupuje po ose od symbolického (deliberativního) pólu směrem k pólu reaktivnímu a končí u hybridních architektur, které se snaží oba póly spojit:

Přístup	Kde je v této kapitole
symbolické (deliberativní) řízení	deduktivní agent, BDI, STRIPS, PRS (odst. 2.1–2.6)
symbolické reaktivní plánování	subsumpční architektura (Brooks), odst. 2.8
konekcionistické reaktivní plánování	agent network architecture (Maes), odst. 2.9
hybridní přístupy	vrstvené architektury, TouringMachines, InteRRaP, IDA, odst. 2.11 a dále

2.1 Klasický přístup UI a deduktivní agent

Klasický způsob, jakým UI staví „inteligentní systém“, stojí na dvou pilířích:

1. **Symbolická reprezentace** prostředí a chování, zde konkrétně reprezentace pomocí logických formulí.
2. **Syntaktická manipulace** s touto reprezentací, tedy logická dedukce neboli dokazování vět.

Teorie o chování agenta je pak zároveň jeho programem. Je to *spustitelná specifikace*, ze které se přímo generují konkrétní akce, a právě v tom je půvab celého přístupu. Naráží ovšem na dva klasické problémy.

- **Problém transdukce** (*transduction problem*) se ptá, jak reálný svět převést do symbolické reprezentace. Spadá sem počítačové vidění, zpracování přirozeného jazyka i učení. Goethem řečeno: „Šedá, milý příteli, je všechna teorie, a zelený je zlatý strom života.“
- **Problém reprezentace a uvažování** se ptá, jak znalosti zapsat a jak nad nimi potom efektivně odvozovat. Odpovědi jsou dokazovače a plánovače.

Odbočka: dedukce vs. indukce. *Dedukce* je odvození závěrů, které jsou pravdivé, jsou-li pravdivé předpoklady; postupuje se při ní od obecného ke konkrétnímu. Školním příkladem je sylogismus „Všichni lidé jsou smrtelní, Sókratés je člověk, tedy Sókratés je smrtelný“. *Indukce* naproti tomu

znamená, že při pravdivých předpokladech je závěr spíše pravdivý než nepravdivý, a postupuje se při ní od konkrétního k obecnému. Sherlock Holmes tak ve skutečnosti používá indukci, i když jí sám říká dedukce. Matematická indukce je navzdory svému jménu naopak dedukce.

Deduktivní agent (agent jako dokazovač vět)

Nechť L je množina formulí predikátové logiky prvního řádu a $D = 2^L$ množina databází těchto formulí. Vnitřní stav agenta tvoří databáze $DB \in D$. Uvažování je realizováno odvozovacími pravidly ρ :

$DB \vdash_{\rho} \varphi$ — formulí φ lze odvodit z DB pomocí pravidel ρ .

Agent je pak trojicí funkcí $see : E \rightarrow Per$, $next : D \times Per \rightarrow D$, $action : D \rightarrow A$, přičemž $action$ je definována procedurou níže.

Výběr akce jako dokazování.

```

1: function ACTION( $DB : D$ ) :  $A$ 
2:   for all  $a \in A$  do
3:     if  $DB \vdash_{\rho} Do(a)$  then return  $a$ 
4:   end if
5: end for
6: for all  $a \in A$  do
7:   if  $DB \not\vdash_{\rho} \neg Do(a)$  then return  $a$ 
8: end if
9: end for
10: return null
11: end function

```

První cyklus hledá akci, kterou lze *dokázat* jako žádoucí, tedy takovou, pro kterou se z databáze odvodí $Do(a)$. Pokud žádná taková akce neexistuje, agent ze svého nároku sleví: druhý cyklus se spokojí s akcí, která je s databází alespoň *konzistentní*, protože o ní nelze dokázat, že by se dělat neměla. Selže-li i to, agent nedělá nic.

Příklad: vysavačový svět 3×3 . Svět popisují tři predikáty: $In(x, y)$ říká, že agent je na pozici (x, y) , $Dirt(x, y)$, že na (x, y) je špína, a $Facing(d)$, že agent je otočen směrem d . Akce se pak odvozují z pravidel:

$$\begin{aligned}
 In(x, y) \wedge Dirt(x, y) &\Rightarrow Do(suck) \\
 In(0, 0) \wedge Facing(north) \wedge \neg Dirt(0, 0) &\Rightarrow Do(forward) \\
 In(0, 2) \wedge Facing(north) \wedge \neg Dirt(0, 2) &\Rightarrow Do(turn) \quad \dots
 \end{aligned}$$

Agent tak systematicky projde celý svět ve tvaru „hada“ 00–01–02–12–11–10–... Stojí za povšimnutí, že úklid má vyšší prioritu než pohyb: pravidlo pro *suck* je uvedeno jako první a všechna ostatní pravidla mají v předpokladu negaci *Dirt*.

Klady a zápory.

- + Přístup je elegantní a má jasnou (a krásnou) sémantiku; specifikace je zároveň programem.
- Pro netriviální úlohy je prakticky nepoužitelný, protože dokazování je pomalé a nemusí vůbec skončit.
- **Problém vypočitatelné racionality** (*calculative rationality*): agent odvodí optimální akci pro ten stav prostředí, jaký byl na začátku odvozování. Prostor se ale mezitím změnilo a vypočtená akce už optimální není. V rychle se měnícím prostředí je to katastrofa.
- Obtížně se navrhuje funkce *see*: není zřejmé, jak z obrázku udělat formule ani jak reprezentovat čas.

Přesto jednotlivé prvky tohoto přístupu žijí dál v pozdějších architekturách, nejvýrazněji v BDI.

2.2 Praktické uvažování: architektura BDI

Deduktivní agent ztroskotál na tom, že se snažil o všem rozhodovat dokazováním. Architektura BDI proto vychází z úplně jiné inspirace, totiž z toho, jak se rozhoduje člověk.

Praktické uvažování (*practical reasoning*)

Je inspirováno lidským rozhodováním. *Teoretické* uvažování (Sókratés) vede jen k tomu, co si o světě myslíme, kdežto *praktické* uvažování vede k **akcím**. Probíhá ve dvou fázích:

- **Deliberace** (*deliberation*) rozhoduje o tom, *čeho chceme dosáhnout* („chci dodělat magisterské studium“).
- **Uvažování o prostředcích** (*means-ends reasoning*), tedy plánování, rozhoduje o tom, *jak toho dosáhneme* (vytvořit plán, jak dostudovat).

A obojí nesmí trvat příliš dlouho.

Beliefs — Desires — Intentions

Beliefs (přesvědčení) tvoří informační stav agenta, tedy jeho znalosti o světě. V MAS jim záměrně neříkáme „znalosti“, abychom zdůraznili, že jsou *subjektivní*, že *nemusí být pravdivé* a že se *mohou v budoucnu změnit*. Mohou obsahovat i odvozovací pravidla umožňující dopředné řetězení.

Desires (touhy) tvoří motivační stav agenta. Jsou to cíle nebo situace, kterých by agent chtěl dosáhnout, a na rozdíl od záměrů *mohou být vzájemně nekonzistentní* (chci se učit i jít na pivo).

Intentions (záměry) jsou stavy světa, kterých se agent *zavázal* dosáhnout. Právě záměry vedou k akcím, protože agent kvůli nim plánuje. Záměry **přetrvávají**, dokud nenastane jedna ze tří věcí:

1. agent jich dosáhne, nebo
2. začne věřit, že jsou nedosažitelné, nebo
3. pominou důvody, které k nim vedly.

Kromě toho záměry *omezují další deliberaci*, protože agent už znovu nezvažuje možnosti neslučitelné s přijatým záměrem, a *ovlivňují budoucí přesvědčení*, protože agent počítá s tím, že se mu záměr podaří.

Rozdíl mezi touhou a záměrem ilustruje Bratman (1990): „Moje touha zahrát si dnes odpoledne basketbal je pouze potenciálním vlivem na moje odpolední chování. Musí soupeřit s mými dalšími relevantními touhami... Naproti tomu jakmile *zamýšlím* hrát dnes odpoledne basketbal, je věc rozhodnuta: obvykle už nemusím dál vážit pro a proti; až odpoledne přijde, prostě svůj záměr vykonám.“

Funkce deliberace

Označme $B \subseteq Bel$ aktuální přesvědčení, $D \subseteq Des$ aktuální touhy a $I \subseteq Int$ aktuální záměry agenta. Deliberace se skládá ze tří funkcí:

- $brf : 2^{Bel} \times Per \rightarrow 2^{Bel}$ je *belief revision function*, která aktualizuje přesvědčení podle vjemu;
- $options : 2^{Bel} \times 2^{Int} \rightarrow 2^{Des}$ generuje možnosti (touhy) z přesvědčení a ze stávajících záměrů;
- $filter : 2^{Bel} \times 2^{Des} \times 2^{Int} \rightarrow 2^{Int}$ z nabídnutých možností vybírá záměry.

Rozdělení deliberace na *generování možností* a *filtrování* je klíčové: generování je levné a široké, zatímco filtrování je vlastní rozhodování, které bere v úvahu i stávající závazky.

2.3 Plánování: STRIPS

Druhou fází praktického uvažování je plánování. Zatímco deliberace vybere cíl, plánování hledá cestu k němu.

Uvažování o prostředcích / plánování

Je to proces rozhodování, jak dosáhnout cíle (záměru) pomocí dostupných prostředků (akcí).

- **Vstup:** cíl (= záměr I); aktuální stav prostředí (= přesvědčení B); akce dostupné agentovi (Ac).
- **Výstup:** plán, tedy posloupnost akcí, po jejímž provedení je cíl splněn.

STRIPS (Fikes, Nilsson, 1971) modeluje svět množinou formulí logiky prvního řádu. Každá akce je popsána *schématem* s předpoklady a efekty, tedy s přidávanými a mazanými fakty. Plánovací algoritmus najde rozdíl mezi cílem a aktuálním stavem a vhodně zvolenou akcí ho zmenší; tomuto postupu se říká *means-ends analysis*. Je to hezké, ale ne příliš praktické, protože algoritmus se často utápí v detailech nízké úrovně.

Deskriptor akce, plánovací problém, plán

Deskriptor akce a je trojice $\langle P_a, D_a, A_a \rangle$, kde P_a jsou předpoklady (*preconditions*), D_a mazaná množina (*delete list*) a A_a přidávaná množina (*add list*); pro jednoduchost jde o uzemněné atomické formule (bez spojek a proměnných).

Plánovací problém je trojice $\langle B_0, O, G \rangle$, kde B_0 jsou počáteční přesvědčení, $O = \{ \langle P_a, D_a, A_a \rangle : a \in Ac \}$ deskriptory všech akcí a G množina formulí popisujících cíl.

Plán $\pi = (a_1, \dots, a_n)$, $a_i \in Ac$, určuje posloupnost databází

$$B_i = (B_{i-1} \setminus D_{a_i}) \cup A_{a_i}, \quad i = 1, \dots, n.$$

Plán je **přípustný** (*admissible*), pokud $B_{i-1} \models P_{a_i}$ pro každé i . Plán je **korektní/validní** (*correct*), pokud je přípustný a navíc $B_n \models G$.

Plánování znamená pro zadaný $\langle B_0, O, G \rangle$ najít korektní plán, nebo prohlásit, že žádný neexistuje.

Příklad: svět kostek (blocks world). Situaci popisují predikáty $On(x, y)$ (x je na y), $OnTable(x)$, $Clear(x)$ (na x nic není), $Holding(x)$ (rameno drží x) a $ArmEmpty$. Agent má k dispozici čtyři akce:

Akce	Pre / Del	Add
$Stack(x, y)$	Pre: $\{Clear(y), Holding(x)\}$ Del: $\{Clear(y), Holding(x)\}$	$\{ArmEmpty, On(x, y)\}$
$UnStack(x, y)$	Pre: $\{On(x, y), Clear(x), ArmEmpty\}$ Del: $\{On(x, y), ArmEmpty\}$	$\{Holding(x), Clear(y)\}$
$Pickup(x)$	Pre: $\{OnTable(x), Clear(x), ArmEmpty\}$ Del: $\{OnTable(x), ArmEmpty\}$	$\{Holding(x)\}$
$PutDown(x)$	Pre: $\{Holding(x)\}$ Del: $\{Holding(x)\}$	$\{ArmEmpty, OnTable(x)\}$

Vyjdeme z počátečního stavu

$$B_0 = \{Clear(A), On(A, B), OnTable(B), OnTable(C), Clear(C), ArmEmpty\}$$

a chceme dosáhnout cíle $G = \{OnTable(A), OnTable(B), OnTable(C)\}$.

Korektní plán je $\pi = (UnStack(A, B), PutDown(A))$. Ověřit se dá krok za krokem:

- Platí $B_0 \models \{On(A, B), Clear(A), ArmEmpty\}$, takže první akce je přípustná; po jejím provedení dostáváme

$$B_1 = (B_0 \setminus \{On(A, B), ArmEmpty\}) \cup \{Holding(A), Clear(B)\}.$$

- Platí $B_1 \models \{Holding(A)\}$, takže i druhá akce je přípustná; po ní dostáváme

$$B_2 = (B_1 \setminus \{Holding(A)\}) \cup \{ArmEmpty, OnTable(A)\}.$$

- Konečně $B_2 \models G$, takže plán je korektní.

Pomocné funkce nad plány

Plan je množina všech plánů nad *Ac*; *pre*(π) je předpoklad plánu; *body*(π) tělo plánu, tedy posloupnost akcí; *empty*(π) se ptá, zda je plán prázdný; *execute*(π) provede všechny akce plánu; *hd*(π) vrací první akci a *tail*(π) zbytek; *sound*(π, I, B) říká, že plán π je korektní vzhledem k záměrům *I* a přesvědčením *B*.

Plánovací funkce agenta má tvar $plan : 2^{Bel} \times 2^{Int} \times 2^{Ac} \rightarrow Plan$.

Knihovna plánů. Konstruovat plány online je drahé, a agent to naštěstí dělat nemusí. Velmi často je proto *plan*() implementována jako *knihovna hotových plánů*: stačí ji jednou projít a zkontrolovat, zda předpoklady plánu odpovídají aktuálním přesvědčením a zda efekty plánu odpovídají cíli. Tento nápad je jádrem architektury PRS (sekce 2.6).

2.4 Implementace BDI agenta

Jednotlivé díly už máme pohromadě: revizi přesvědčení, deliberaci i plánování. Teď je stačí složit do jedné řídicí smyčky.

Hlavní řídicí smyčka BDI agenta:

```

1:  $B \leftarrow B_0; \quad I \leftarrow I_0$ 
2: while true do
3:    $v \leftarrow see()$ 
4:    $B \leftarrow brf(B, v)$ 
5:    $D \leftarrow options(B, I)$ 
6:    $I \leftarrow filter(B, D, I)$ 
7:    $\pi \leftarrow plan(B, I, Ac)$ 
8:   while  $\neg(empty(\pi) \vee succeeded(I, B) \vee impossible(I, B))$  do
9:      $a \leftarrow hd(\pi); \quad execute(a); \quad \pi \leftarrow tail(\pi)$ 
10:     $v \leftarrow see(); \quad B \leftarrow brf(B, v)$ 
11:    if reconsider(I, B) then
12:       $D \leftarrow options(B, I); \quad I \leftarrow filter(B, D, I)$ 
13:    end if
14:    if  $\neg sound(\pi, I, B)$  then
15:       $\pi \leftarrow plan(B, I, Ac)$  ▷ přeplánování
16:    end if
17:  end while
18: end while
```

Dva predikáty ve vnitřním cyklu si zaslouží vysvětlení. Predikát *succeeded*(*I*, *B*) říká, že záměry *I* platí za předpokladu přesvědčení *B*, kdežto *impossible*(*I*, *B*) říká, že záměry *I* za předpokladu *B* platit nemohou. Vnitřní cyklus tedy skončí ve třech případech: když je plán vyčerpán, když je cíl splněn, anebo když se cíl stal nedosažitelným.

2.5 Odhodlání agenta (*commitment*)

Strategie odhodlání vůči záměrům

Kdy má agent opustit cíl? Nabízejí se tři odpovědi:

- **Slepé odhodlání** (*blind commitment*, též *fanatical*) drží záměr tak dlouho, dokud agent *nezačne věřit, že ho dosáhl*. Takový agent se nikdy nevzdá.
- **Jednosměrné odhodlání** (*single-minded commitment*) drží záměr, dokud agent *nezačne věřit, že ho dosáhl, nebo že už není dosažitelný*.
- **Otevřené odhodlání** (*open-minded commitment*) nechává záměr přetrvávat tak dlouho,

dokud agent věří, že je stále dosažitelný a stále chtěný. Záměr proto může být opuštěn i při pouhé změně motivace.

Odlišit je ale potřeba **odhodlání vůči plánu** od **odhodlání vůči cíli**. Agent v algoritmu výše je odhodlán vůči jednomu plánu a opustí ho tehdy, když věří, že cíl je splněn, že je nedosažitelný, anebo když je plán prázdný. Zbývá tím ovšem ta těžší otázka: kdy má agent *přehodnotit sám záměr*?

Klasické dilema zní, že deliberace **trvá čas** a prostředí se během ní mění. Obě krajní polohy jsou špatně.

- Agent, který záměry *nepřehodnocuje*, se může upnout k cílům, jichž už nelze dosáhnout.
- Agent, který přehodnocuje *příliš často*, je zase tak zaneprázdněn uvažováním, že nic nestihne.

Východiskem je **metařízení** (Wooldridge, Parsons, 1999). Zavádí se funkce *reconsider()*, která je výpočetně jednodušší než samotné přehodnocení a jen odhaduje, zda se přehodnocení vyplatí. Krite-rium kvality je přirozené: *reconsider()* funguje dobře, pokud pokaždé, když navrhne přehodnocení, agent po deliberaci *skutečně* záměr změní, a naopak.

Odvážný vs. opatrný agent

Odvážný agent (*bold agent*) přehodnocuje záměry teprve po dokončení celého plánu; kvůli deliberaci plán nikdy nepřeruší.

Opatrný agent (*cautious agent*) přehodnocuje po každé akci plánu.

Míra odvahy (*level of boldness*) udává, kolik akcí agent provede mezi dvěma deliberacemi.

Dynamičnost prostředí (*rate of world change*) udává, kolikrát se prostředí změní během jednoho cyklu agenta.

Efektivita agenta = počet dosažených záměrů / počet všech záměrů.

Experimentální výsledek (Kinny, Georgeff, v Tileworldu): který z obou typů agentů vyhraje, závisí na prostředí.

- Mění-li se svět *pomalou*, jsou efektivnější **odvážní** agenti, protože deliberace by pro ně byla jen zbytečnou režií.
- Mění-li se svět *rychle*, překonávají je **opatrní** agenti, protože plány rychle zastarávají.

Je to konkrétní kvantitativní podoba obecného kompromisu reaktivita vs. proaktivita.

2.6 Procedurální odvozovací systém (PRS)

PRS — *Procedural Reasoning System*

Vznikl v 80. letech na Stanfordu (Georgeff a kol.). Jde o **první implementaci architektury BDI** a zároveň o jednu z prakticky nejúspěšnějších agentních architektur vůbec; byla mnohokrát reimplementována jako **AgentSpeak/Jason**, **JAM**, **JACK** nebo **Jadex**.

Z praktických nasazení jmenujme OASIS (systém řízení letového provozu v Sydney), SPOC (organizace podnikových procesů) a SWARMM (letecký simulátor australského letectva).

Komponenty PRS agenta. Agent se skládá z databáze *beliefs* (atomy FOL prologovského typu), z *desires* (cíle), z *knihovny plánů* (*plan library*), ze zásobníku *intentions* a z *interpretu*, který vše propojuje. Data ze senzorů vstupují do beliefs a interpret na druhém konci vydává akce.

Plán v PRS

Agent má knihovnu hotových plánů, které reprezentují jeho *procedurální znalosti*. **Neplánuje se od nuly**, pouze se z knihovny *vybírá*. Plán se skládá ze tří částí:

- **Goal** (též *invocation condition* či *cue*) je podmínka, která má platit po vykonání plánu; určuje, na jaký cíl se plán hodí.
- **Context** je podmínka nutná pro spuštění plánu a musí být konzistentní s přesvědčeními.

- **Body** jsou akce, které se mají vykonat.

Tělo plánu *není* jen lineární posloupnost akcí, protože může obsahovat i další *cíle* tvaru „dosáhni *f*“, „dosáhni *f* nebo *g*“ nebo „dosahuj *f*, dokud nenastane *g*“. Tím vzniká hierarchické rozkládání cílů na podcíle.

Běh interpretu probíhá takto:

1. Nejprve se agent inicializuje počátečními přesvědčeními B_0 a cílem nejvyšší úrovně (*top-level goal*).
2. Zásobník záměrů obsahuje aktuální cíle v různých stavech rozpracovanosti.
3. Interpret vezme záměr z vrcholu zásobníku a v knihovně plánů hledá plány s odpovídajícím cílem.
4. Z nalezených plánů vybere ty, jejichž *kontext* je konzistentní s aktuálními přesvědčeními; ty tvoří aktuální *options* neboli *desires*.
5. **Deliberace** je pak výběr jednoho záměru z těchto možností. Nabízejí se dvě cesty:
 - Původní PRS používal *metaplány*, tedy plány o plánech, které modifikovaly záměry agenta. Ukázalo se to jako příliš složité.
 - Praktičtější varianta dává každému plánu číselné *ohodnocení* (očekávanou utilitu) a vybere plán s nejvyšší hodnotou.
6. Vybraný plán se vykoná, což může vést k přidání dalších záměrů na zásobník.
7. Selže-li plán, agent zvolí jiný záměr z možností a pokračuje.

Ukázka — Jason/AgentSpeak (agent nosí majiteli pivo z ledničky):

```
available(beer,fridge). limit(beer,10). /* počáteční beliefs */
too_much(B) :- .count(consumed(_,_ ,B),Q) & limit(B,L) & Q > L. /* pravidlo */
+!has(owner,beer) /* spouštěcí událost */
: available(beer,fridge) & not too_much(beer) /* kontext */
<- !at(robot,fridge); open(fridge); get(beer); close(fridge); /* tělo */
!at(robot,owner); hand_in(beer).
+!at(robot,P) : not at(robot,P) <- move_towards(P); !at(robot,P).
```

Zápis `+!g : c <- b` se čte: „nastane-li událost přidání cíle *g* a platí-li kontext *c*, vykonej tělo *b*“. Symbol `!` označuje *achievement goal* (dosáhni), symboly `+` a `-` přidání a odebrání přesvědčení nebo cíle a `.send` a `.count` jsou vestavěné akce.

2.7 Reaktivní přístup a jeho východiska

V 80. a 90. letech se ukázalo, že drobnými úpravami symbolického přístupu se jeho potíže odstranit nedají. Vznikla proto alternativní paradigmata UI, která se od symbolické tradice neodchýlila v detailu, ale rovnou v základu. Jejich společným jmenovatelem je **odmítnutí symbolické reprezentace** i uvažování postaveného na syntaktické manipulaci s ní. Opřít se dají o tři klíčové teze.

Tři principy reaktivního přístupu

Interakce (*situatedness*). Inteligentní chování závisí na prostředí, ve kterém je agent *zasazen*, a nelze ho proto studovat izolovaně od tohoto prostředí.

Ztělesnění (*embodiment*). Inteligentní chování není jen logika: je produktem agenta, který *má tělo* a fyzicky interaguje se světem.

Emergence. Inteligentní chování *vzniká* (emerguje) interakcí mnoha jednoduchých chování a není nikde explicitně naprogramováno.

Z těchto tezí plynou i charakteristiky reaktivních agentů. Jsou *behaviorální*, protože těžiště návrhu leží ve vývoji a kombinaci jednotlivých chování, a právě jejich repertoár agenta definuje. Jsou *situované*, tedy neoddelitelné od svého prostředí. A jsou *reaktivní*: agent v podstatě jen reaguje na prostředí a nedeliberuje. Tomu odpovídá i reprezentace, se kterou pracuje, protože ta je **subsymblická**. Bývají to konekcionistické sítě, konečné automaty nebo jednoduchá pravidla typu IF–THEN.

2.8 Symbolické reaktivní plánování: subsumpční architektura

Reaktivní program dotáhl nejdál robotik, který chtěl ukázat, že se i poměrně složité chování obejde bez modelu světa. Konflikty mezi chováními u něj rozhoduje pevně dané pořadí priorit.

Subsumpční architektura (*subsumption architecture*)

Autorem je Rodney Brooks, 1986–1991, a jde o nejúspěšnější reaktivní přístup vůbec. Vychází ze tří Brooksových tezí:

1. Inteligentní chování lze generovat *bez symbolické reprezentace*.
2. Inteligentní chování lze generovat *bez explicitního abstraktního uvažování*.
3. Intelligence je *emergentní vlastnost* určitých složitých systémů.

Intelligence agenta je realizována množinou jednoduchých cílově orientovaných **chování** (*behaviours*), pro něž platí:

- Každé chování je samo o sobě mechanismem výběru akce, totiž dvojicí *situace* $\rightarrow$ *akce*.
- Každé chování přijímá vjemy a transformuje je na akce. Odpovídá vždy za jeden dílčí cíl a má jednoduchou strukturu pravidla.
- Chování běží *paralelně* a *soupeří* spolu o kontrolu nad agentem.
- Chování jsou uspořádána do **subsumpční hierarchie**, která definuje jejich priority: vyšší vrstva nižší „pohlcuje“, tedy potlačuje ji.

Mechanismus výběru akce v subsumpční architektuře:

```
1: function ACTION(vjem  $p$ )  
2:    $fired \leftarrow \{ b \in Behaviours \mid p \text{ splňuje podmínku } b \}$  ▷ chování, která „vystřelí“  
3:   for all  $b \in fired$  do  
4:     if neexistuje  $b' \in fired$  takové, že  $b' \prec b$  then ▷  $b$  není nikým inhibováno  
5:       return akce příslušná chování  $b$   
6:     end if  
7:   end for  
8:   return null ▷ nevystřelilo nic, agent nedělá nic  
9: end function
```

Formálně je chování dvojice $\langle c, a \rangle$, kde $c \subseteq Per$ je podmínka a $a \in A$ akce. Na množině chování R je definována relace inhibice $\prec \subseteq R \times R$, která tvoří striktní totální uspořádání. Pozor na to, kterým směrem se relace čte, protože právě tady se dá snadno chybovat. U Wooldridge i na přednášce znamená $b_1 \prec b_2$, že b_1 **inhibuje** b_2 , takže b_1 leží *níže* v subsumpční hierarchii a má *přednost*. Zápis $b_1 \prec b_2 \prec \dots \prec b_n$ proto vyjmenovává chování od nejvyšší priority k nejnižší.

Celý mechanismus má dvě přednosti. Je *jednoduchý*, byť při desítkách chování se nastavení priorit stává záluďným, a je *velmi rychlý*: dá se implementovat hardwarově a má konstantní složitost.

Příklad: Steelsův průzkumník Marsu. Cílem je sbírat vzácné vzorky hornin na vzdálené planetě, a to rojem robotických průzkumníků. Základna k tomu vysílá navigační signál, takže roboti nemusí umět nic víc než detekovat *gradient* tohoto signálu. Každý robot navíc veze radioaktivní drošky, kterými *nepřímo komunikuje* s ostatními, tedy pomocí stigmergie. Přímá komunikace tak vůbec není potřeba.

První iterace zvládne jen náhodnou procházku a návrat jednoho robota. Priorita pravidel klesá v tabulce shora dolů:

R1	if vidím překážku then změň směr
R2	if nesu vzorek and jsem na základně then polož vzorek
R3	if nesu vzorek and nejsem na základně then jdi po gradientu nahoru
R4	if vidím vzorek then seber ho
R5	if true then pohybuj se náhodně

Druhá iterace zlepší průzkum pomocí stigmergie. Vzorky se totiž často vyskytují ve shlucích, a tak robot, který jeden našel, o tom ostatním „řekne“ tím, že cestou domů rozhazuje drobký:

```

R6  if nesu vzorek and jsem na základně then polož vzorek
R7  if nesu vzorek and nejsem na základně then polož 2 drobký a jdi po gradientu nahoru
R8  if cítím drobek then seber 1 drobek a jdi po gradientu dolů

```

Priority vypadají v notaci „levé inhibuje pravé“, tedy seřazeny od nejvyšší priority, takto: $R1 \prec R6 \prec R7 \prec R4 \prec R8 \prec R5$. Robot, který sleduje stopu, z ní bere vždy po jednom drobký, takže se stopa postupně *vypaří*, jakmile je shluk vytěžen. Roj tedy vykazuje kooperativní chování, přestože žádný jednotlivý robot nemá model světa ani neposílá zprávy.

2.9 Konekcionistické reaktivní plánování: agent network architecture

Subsumpční architektura řeší konflikty *pevnou* prioritou, a to je rigidní. Konekcionistický přístup proto pevné priority nahradí **spojitou aktivací šířenou sítí**, takže výběr akce vzejde z dynamiky sítě, a ne z pořadí zapsaného předem. Výsledku se říká „reaktivní plánování“: agent se chová, jako by plánoval, ale žádný explicitní plán nikdy nevytvoří.

Agent network architecture / spreading activation networks

Autorkou je Pattie Maes, 1989–1991. Agent je množina **kompetenčních modulů** (*competence modules*), které se podobají Brooksovým chováním. Každý modul i má:

- **předpoklady** c_i (*preconditions*), tedy to, co musí platit, aby byl modul *proveditelný* (*executable*);
- **efekty**, kterými jsou *add list* a_i a *delete list* d_i , stejně jako je tomu u STRIPS;
- **úroveň aktivace** α_i a globální **práh aktivace** θ .

Moduly jsou propojeny v orientovaném grafu podle svých podmínek: hrany tvoří shoda pre- a postpodmínek a k nim přibývají hrany reprezentující časovou následnost a konflikty. V každém kroku se aktivace šíří a **vybrán je proveditelný modul s nejvyšší aktivací**, pokud jeho aktivace překročí práh θ .

Aktivace přitéká do sítě ze tří zdrojů:

- **Ze stavu světa** (*state*) ji dostávají moduly, jejichž předpoklady jsou momentálně splněny, tedy ty, o nichž platí „teď se dá udělat tohle“.
- **Z cílů** (*goals*) ji dostávají moduly, jejichž *add list* obsahuje cíl, tedy ty, o nichž platí „tohle vede k cíli“.
- **Z ochranných cílů** (*protected goals*) naopak plyne *inhibice*, a to u modulů, jejichž *delete list* obsahuje již splněný cíl. O těch totiž platí „tohle by zbořilo, co už mám“.

Mezi moduly navzájem se pak aktivace šíří třemi typy spojů:

- **Následnické spoje** (*successor links*) vedou od modulu x k modulu y tehdy, když y přidává něco, co x potřebuje jako předpoklad. Jde o *dopředné* řetězení ve smyslu „až budu hotov, půjde to dál“.
- **Předchůdcovské spoje** (*predecessor links*) používá modul x , který není proveditelný: posílá aktivaci modulům, jež by mu splnily chybějící předpoklad. Jde o *zpětné* řetězení ve smyslu „pomoz mi, ať můžu běžet“.
- **Konfliktní spoje** (*conflict links*) znamenají, že modul x *inhibuje* modul y , který ruší jeho předpoklady.

Chování celé sítě řídí pět globálních parametrů. Práh aktivace θ je jediným prahem v systému, ϕ je aktivace vpravená do sítě stavem světa, γ aktivace vpravená cíli, δ aktivace odebraná chráněnými cíli a π *průměrná* úroveň aktivace, na kterou se síť po každém kroku normalizuje, aby se celková energie neměnila. Dva z parametrů se dají číst přímo jako návrhové kompromisy. Poměr γ/ϕ přesně vyjadřuje kompromis **proaktivita vs. reaktivita**, zatímco θ určuje **uvážlivost vs. rychlost**: vysoký práh

→ agent dále váhá, ale rozhoduje se lépe. Pokud v daném kroku práh nepřekročí žádný proveditelný modul, sníží se θ o 10 % a zkouší se znovu. Sít tak nemůže „zamrznout“.

Jak se aktivace v jednom kroku počítá. Aktualizace probíhá ve čtyřech fázích. Nejprve přijde na řadu (1) *vstup z prostředí*, kdy každý pravdivý fakt rozdělí kvantum ϕ rovným dílem mezi moduly, které jej mají ve svých předpokladech. Následuje (2) *vstup od cílů*: každý cíl rozdělí γ mezi moduly, které ho přidávají, a každý chráněný cíl naopak *odebere* δ modulům, jež by ho rušily. Ve třetí fázi nastane (3) *šíření mezi moduly*. Proveditelný modul posílá část své aktivace dopředu následníkům, opět rozdělenou rovným dílem mezi příjemce, neproveditelný modul ji posílá zpět modulům, které mu mohou splnit chybějící předpoklad, a konfliktní spoje odpovídající díl aktivace odečítají. Krok uzavírá (4) *normalizace*, při níž se aktivace všech modulů přeškálují tak, aby jejich průměr byl π , a celková energie sítě proto zůstává konstantní. Teprve poté se vybere proveditelný modul s aktivací nad prahem θ ; provedený modul svou aktivaci vynuluje.

Poznámka k přednášce: slidy uvádějí práh aktivace jako vlastnost jednotlivého modulu a mluví o něm jako o prioritě. V Maesově původním modelu je ovšem práh θ *globální* a roli „priority“ hraje aktuální úroveň aktivace α_i , která se dynamicky mění.

Srovnání se subsumpcí: pořadí chování zde není pevné, ale vzniká *dynamicky* z aktuálního stavu světa a z cílů. Sít proto dokáže vygenerovat i víceukrokové „plánovité“ sekvence, protože se aktivace řetěží zpět od cíle přes předchůdcovské spoje, a přitom nikdy nevznikne explicitní datová struktura plánu. Odtud pochází název *reaktivní plánování*. Cenou za to je citlivost na nastavení parametrů a nemožnost zaručit optimalitu či úplnost.

2.10 Omezení reaktivních architektur

Reaktivní architektury si za svou rychlost a jednoduchost platí, a to hned ve čtyřech ohledech:

- Reaktivní agenti **nemají model světa** a všechno musí odvodit z aktuálních vjemů. Někdy proto musí *měnit prostředí*, aby si informaci „zapamatovali“. Přesně to dělají radioaktivní drošky, které fungují jako externalizovaná paměť, tedy jako stigmergie.
- Mají jen **krátkodobý pohled**, protože jednají podle aktuálního stavu a podle lokální informace. Zohlednit globální podmínky a dlouhodobé cíle je pak obtížné.
- Emergence **není dobrý inženýrský postup**: chování je nutné vyladit experimentálně a chybí k tomu jak metodologie, tak možnost verifikace.
- Návrh je zvládnutelný pro několik vrstev, ale s **desítkami chování se stává neřešitelným**, protože počet interakcí mezi nimi roste kvadraticky.

2.11 Hybridní architektury

Ani čistě reaktivní, ani čistě deliberativní architektura tedy není ideální. Nabízí se proto obojí zkombinovat, a právě to dělají hybridní architektury:

- **Deliberativní neboli plánovací komponenta** pracuje na symbolické úrovni a vytváří reprezentace a plány.
- **Reaktivní komponenta** zajišťuje okamžité reakce bez složitých výpočtů.

Komponenty bývají uspořádány do **vrstev** (*layered architectures*) a reaktivní vrstvy mají zpravidla *přednost* před deliberativními, protože bezpečnost je důležitější než optimalita. Uspořádat vrstvy vůči senzorům a efektorům jde dvěma způsoby.

Horizontální vs. vertikální vrstvení

Horizontální vrstvení: všechny vrstvy jsou *paralelně* připojeny k senzorům i k efektorům. Každá vrstva se chová jako samostatný agent a navrhuje akci.

+ Konceptně je to jednoduché, protože pro n druhů chování stačí n vrstev.

– Vrstvy se navzájem ovlivňují a jejich návrhy si odporují, takže je nutná **mediační funkce**

(*mediator function*), která konflikt rozhodne. Právě ta je úzkým hrdlem: musí uvažovat m^n kombinací, kde n je počet vrstev a m počet možných návrhů, a zároveň je kritickým bodem selhání.

Vertikální vrstvení: vrstvy jsou zapojeny *sériově*.

- Ve variantě *one-pass* vjem vstoupí do nejnižší vrstvy, prochází nahoru a akce vystupuje z nejvyšší. Uspořádání i hierarchie chování jsou tak přirozené.
 - Ve variantě *two-pass* putují vjemy *zdola nahoru* a řídicí povely, tedy akce, *shora dolů*. Tok připomíná řízení ve skutečné firmě.
- + Odpadá potřeba mediace a složitost interakcí je lineární, protože rozhraní je jen $n - 1$.
– Chybí robustnost: selhání kterékoli vrstvy shodí celého agenta.

TouringMachines (Ferguson, 1992)

Horizontální třívrstvá architektura pro simulovaného agenta v dopravním prostředí.

- **Modelovací vrstva** (*modelling layer*) drží symbolické modely agenta, prostředí a ostatních agentů. Předvídá a řeší konflikty, stanovuje cíle a posílá je plánovací vrstvě.
- **Plánovací vrstva** (*planning layer*) zajišťuje proaktivní chování: vybírá z předpřipravených plánů podobně jako PRS a sestavuje z nich plán cesty.
- **Reaktivní vrstva** (*reactive layer*) obsahuje klasická reaktivní pravidla *situace* → *akce* a stará se o rychlou okamžitou reakci, třeba o vyhýbání překážkám.

Vrstvy pracují paralelně a jejich výstupy sjednocuje sada **řídících pravidel** (*control rules*), což je konkrétní podoba mediační funkce. Tato pravidla mohou vstupy a výstupy vrstev potlačovat: slouží k tomu pravidla *sensor* a *suppressor*.

InteRRaP (Müller, 1994)

Vertikální dvouprůchodová (*two-pass*) třívrstvá architektura.

- **Vrstva kooperativního plánování** (*cooperative planning layer*) obstarává sociální interakce a společné plány s ostatními agenty.
- **Vrstva lokálního plánování** (*local planning layer*) zajišťuje běžné individuální plánování.
- **Behaviorální vrstva** (*behaviour-based layer*) obsahuje reaktivní chování.

Každá vrstva má *vlastní znalostní bázi* na příslušné úrovni abstrakce, tedy znalosti o světě, plány, případně společné plány a modely ostatních agentů. Mezi sebou vrstvy komunikují dvěma operacemi: *aktivací zdola nahoru*, kdy nižší vrstva situaci nezvládne a předá ji výš, a *zásahem shora dolů*, kdy vyšší vrstva používá nižší k vykonání svých rozhodnutí.

Reálný příklad: Stanley (Volkswagen Touareg R5, Stanford). Tento autonomní vůz vyhrál DARPA Grand Challenge 2005, tedy závod na 212 km Mohavskou pouští, a je přímým předchůdcem dnešních autonomních vozidel. Jeho architektura je vrstvená:

- vrstva senzorů → vrstva abstraktní percepce → vrstva plánování a řízení, která sestaví plán trasy a řídí vůz, → vrstva rozhraní vozidla;
- k tomu přistupuje vrstva uživatelského rozhraní (panel, start) a vrstva globálních služeb (soutbový systém, komunikace, hodiny).

2.12 Komplexní architektury: IDA a globální pracovní prostor

Poslední rodina architektur jde opačným směrem než reaktivní minimalismus. Nesnaží se agenta osekát na nezbytné minimum, ale integrovat do něj *všechno*: výběr akce, paměť, deliberaci i emoce.

IDA — *Intelligent Distribution Agent* (S. Franklin)

Jedna z nejsložitějších agentních architektur, jaké kdy byly postaveny. Vznikla pro reálnou úlohu amerického námořnictva, totiž pro přidělování personálu na nová služební místa, odkud pochází ono „distribution“; distribuovaná je ostatně i sama architektura.

Východiskem je teorie globálního pracovního prostoru (*global workspace theory*, Baars, 1988–97), jedna z teorií vědomí. Stojí na těchto předpokladech:

- Mysl je multiagentní systém, protože se skládá z mnoha jednoduchých, převážně nevědomých procesů, které provádějí specializované operace.
- Procesy spolu komunikují *zřídka* a pouze přes sdílenou paměť typu *blackboard*, organizovanou jako asociativní pole (srov. kap. 5).
- Procesy dynamicky vytvářejí **koalice** vyšších řádů.
- **Mechanismus výběru akce** spočívá v tom, že koalice, která nejlépe odpovídá aktuálním vstupům, se dostane do „vědomí“, tedy do globálního pracovního prostoru, a je vykonána.
- Existuje *hierarchie kontextů* reprezentujících svět na různých úrovních: kontext cílů, vjemů, smyslů, konceptů a kontext kulturní.

Hodnocení: architektura kombinuje mnoho přístupů z MAS i ze zbytku UI, tedy výběr akce, paměť, deliberaci a emoce, a právě v tom je zároveň její slabina. Řada rozhodnutí je totiž *ad hoc* a sladit tak heterogenní kolekci modelů do optimálně spolupracujícího celku je velmi obtížné.

Při srovnávání mechanismů výběru akce je IDA užitečný jako *čtvrtý* typ arbitráže. Subsumpce rozhoduje **pevnou prioritou**, síť Maes **úrovní aktivace**, vrstvené architektury **mediační funkcí** a IDA **soutěží dynamicky vznikajících koalic** o přístup do globálního pracovního prostoru.

2.13 Srovnání architektur

Následující tabulka staví vedle sebe všechny rodiny architektur probrané v této kapitole a ukazuje, že jednotlivé vlastnosti se vzájemně vyvažují: co jedna rodina získá na rychlosti, ztrácí na proaktivitě, a naopak.

	Deduktivní	BDI / PRS	Reaktivní	Hybridní
Reprezentace	symbolická (FOL)	symbolická (beliefs, plány)	subsymbolická / pravidla	smíšená po vrstvách
Výběr akce	dokazování $Do(a)$	deliberace + plán z knihovny	priorita / aktivace	vrstvy + arbitráž
Model světa	úplný	částečný, revidovatelný	žádný	po vrstvách
Rychlost	velmi nízká	střední	vysoká (konst.)	vysoká v reflexu
Reaktivita	špatná	střední (laditelná)	výborná	dobrá
Proaktivita	dobrá	výborná	žádná	dobrá
Návrh	obtížný, ale formální	systematický	experimentální	složitý

2.14 Shrnutí ke zkoušce

- **Deduktivní agent** pracuje s databází $DB \in 2^L$ a akci vybírá dokazováním $DB \vdash_{\rho} Do(a)$. Výběr akce je dvoufázový: hledá se nejprve dokazatelná akce, pak akce konzistentní, a když neuspěje ani to, agent neudělá nic. Přístup je elegantní, ale nepraktický, protože naráží na problém transdukce a na vypočitatelnou racionalitu.
- **BDI** má dvě fáze. První je deliberace, kterou tvoří *brf*, *options* a *filter*, druhou means-ends reasoning, tedy plánování. Beliefs jsou subjektivní a omylné, desires mohou být nekonzistentní a intentions jsou závazky, které přetrvávají a omezují další deliberaci.
- **STRIPS** popisuje akci deskriptorem $\langle P_a, D_a, A_a \rangle$, plánovací problém trojicí $\langle B_0, O, G \rangle$ a stav aktualizuje předpisem $B_i = (B_{i-1} \setminus D_{a_i}) \cup A_{a_i}$. Plán je *přípustný*, pokud platí $B_{i-1} \models P_{a_i}$, a *korektní*, pokud navíc $B_n \models G$. U zkoušky je třeba umět projít svět kostek.
- Zvlášť dobře je potřeba znát **BDI smyčku** a roli funkcí *reconsider* a *sound*.
- **Commitment** má tři podoby: blind, single-minded a open-minded. Podle ochoty přehodnocovat se agenti dělí na bold a cautious, efektivita se počítá jako podíl dosažených a všech záměrů a platí, že v pomalém prostředí vyhrává bold agent, kdežto v rychlém cautious.
- **PRS** je první implementací BDI. Plán se skládá z částí Goal, Context a Body, místo plánování

se používá knihovna plánů a záměry drží zásobník. Deliberace se řeší metaplány nebo utilitou a potomky systému jsou AgentSpeak/Jason, JAM, JACK a Jadex.

- Reaktivní přístup stojí na trojici *situatedness* — *embodiment* — *emergence* a odmítá symbolickou reprezentaci.
- **Subsumpce** (Brooks) představuje *symbolické* reaktivní plánování. Množina jednoduchých chování *situace* → *akce* běží paralelně a je uspořádána do pevné priority. U zkoušky je potřeba umět Steelsova průzkumníka Marsu a roli stigmergie, tedy drobků.
- **Agent network architecture** (Maes) je naopak *konekcionistické* reaktivní plánování. Moduly mají předpoklady a efekty (add, delete), aktivace se šíří následnickými, předchůdcovskými a konfliktními spoji a jejichmi zdroji jsou stav světa, cíle a inhibice od chráněných cílů. Vybere se proveditelný modul s nejvyšší aktivací nad prahem a poměr parametrů γ/ϕ ladí proaktivitu vůči reaktivitě.
- Omezení reaktivních agentů jsou čtyři: nemají model světa, mají jen krátkodobý pohled, jejich návrh není inženýrský a množství vrstev se neškáluje.
- **Hybridní** architektury se dělí na horizontální a vertikální. U horizontálních běží vrstvy paralelně a rozhoduje mediátor, který musí uvažovat m^n kombinací, a je proto úzkým hrdlem. Vertikální jsou sériové ve variantě one-pass nebo two-pass, vystačí si s $n - 1$ rozhraními, ale jsou křehké.
- **TouringMachines** jsou horizontální 3 vrstvy (modelling, planning, reactive) doplněné řídicími pravidly, kdežto **InterRaP** je vertikální two-pass se 3 vrstvami (kooperativní plánování, lokální plánování, behaviorální) a s vlastní KB na každé vrstvě. **Stanley** slouží jako příklad reálného nasazení.
- **IDA** (Franklin) staví na *global workspace theory* (Baars) a chápe mysl jako MAS jednoduchých nevědomých procesů, které komunikují přes blackboard. Koalice procesů soutěží o vstup do „vědomí“ a vítězná se vykoná, přičemž existuje hierarchie kontextů. Jde o nejsložitější architekturu a je v ní mnoho ad hoc rozhodnutí.
- Arbitráž mezi chováními má čtyři typy: **pevnou prioritu** u subsumpce, **úroveň aktivace** u Maes, **mediační funkci** u vrstvených architektur a **soutěž koalic** u IDA.

3 Problém hledání cesty

[♦ nad rámec slidů: celá kapitola] Přednáška NAIL106 řadí plánování i robotiku mezi témata, která *ne pokrývá*, a ve slidech ke hledání cesty proto nic není. Okruh ho ale výslovně jmenuje, takže se mu vyhnout nejde. Kapitola je držena na minimu, které u zkoušky obstojí: algoritmus A* i s vlastnostmi heuristiky, a pak už jen to, co je na hledání cesty vysloveně *multiagentního*, tedy dynamické prostředí a koordinace více agentů.

3.1 Formulace a zařazení

Hledání cesty (*pathfinding*) je nejčastější konkrétní podúloha, kterou musí situovaný agent řešit. Objevuje se v plánovací vrstvě hybridní architektury, vystupuje jako akce `move_towards(P)` v BDI plánu a agent ji také může nabízet ostatním jako službu. V multiagentním kontextu k běžnému zadání přibývají dvě specifika. Prostředí je **dynamické**, tedy mapa se za běhu mění, a ostatní agenti jsou **pohyblivé překážky**, se kterými se agent musí koordinovat.

Problém hledání cesty

Je dán ohodnocený graf $G = (V, E)$ s nezáporným ohodnocením hran $w : E \rightarrow \mathbb{R}_0^+$, počáteční vrchol $s \in V$ a cílový vrchol $g \in V$. Hledáme cestu $s = v_0, v_1, \dots, v_k = g$ minimalizující $\sum_i w(v_{i-1}, v_i)$.

Graf přitom obvykle není vypsáný, ale zadáný *implicitně* nástupnickou funkcí; typicky jde o 4- nebo 8-souvislou mřížku, o navigační síť nebo o graf viditelnosti.

Odkud se vezmou vrcholy. Prostředí je spojitě, kdežto prohledávání potřebuje graf. Musí tedy proběhnout diskretizace a právě její volba rozhoduje o kvalitě výsledné cesty víc než volba samotného algoritmu. Nabízí se několik možností:

- **Mřížka** (4- nebo 8-souvislá) se implementuje triviálně, ale cesta po ní vychází „schodovitě“ a vrcholů je obrovské množství.
- **Graf viditelnosti** má za vrcholy rohy mnohoúhelníkových překážek a jeho hrany spojují dvojice vzájemně viditelných rohů. V rovině s mnohoúhelníkovými překážkami dá *skutečně optimální* cestu, jenže jeho velikost roste kvadraticky s počtem rohů.
- **Navigační síť** (*navmesh*) pokryje prostor konvexními plochami, uvnitř nichž je pohyb volný. Je to standard v herních enginech a v robotice.
- **Stavová mřížka** bere jako vrchol nejen polohu, ale i orientaci a případně rychlost. Bez toho se neobejdou neholonomní vozidla, která se nemohou otáčet na místě.
- **Vzorkovací metody** graf nestaví celý předem. *PRM* (*probabilistic roadmap*) si předpočítá graf náhodných konfigurací, zatímco *RRT* a *RRT** nechávají strom inkrementálně růst. Pro vysokodimenzionální konfigurační prostory, jako je konfigurační prostor robotického ramene, jsou jedinou praktickou volbou.
- **Potenciálová pole** nechají cíl přitahovat a překážky odpuzovat, agent pak jde po gradientu. Jsou velmi levná a reaktivní, takže patří spíš do reaktivní vrstvy (§2.8), zato *uvíznou v lokálním minimu* a nedávají žádnou záruku úplnosti.

3.2 A*

Prohledávání typu *best-first* si ke každému vrcholu n vede tři hodnoty:

$$\begin{aligned} g(n) &= \text{cena nejlepší dosud nalezené cesty } s \rightarrow n, \\ h(n) &= \text{heuristický odhad ceny } n \rightarrow g_{\text{cil}}, \\ f(n) &= g(n) + h(n) \quad \text{— odhad ceny nejlepší cesty } s \rightarrow g_{\text{cil}} \text{ vedoucí přes } n. \end{aligned}$$

Algoritmus vždy expanduje ten z otevřených vrcholů, který má nejmenší f ; narazí-li přitom na levnější cestu k vrcholu, který už zná, jeho hodnoty opraví.

Algoritmus A* (Hart, Nilsson, Raphael, 1968):

```

1:  $OPEN \leftarrow \{s\}$  (prioritní fronta podle  $f$ ),  $CLOSED \leftarrow \emptyset$ 
2:  $g(s) \leftarrow 0$ ,  $f(s) \leftarrow h(s)$ , ostatní  $g \leftarrow \infty$ 
3: while  $OPEN \neq \emptyset$  do
4:    $n \leftarrow \arg \min_{x \in OPEN} f(x)$ ; odeber  $n$  z  $OPEN$ 
5:   if  $n$  je cíl then return rekonstruovaná cesta
6:   end if
7:   přidej  $n$  do  $CLOSED$ 
8:   for all  $m \in succ(n)$  do
9:      $g_{\text{new}} \leftarrow g(n) + w(n, m)$ 
10:    if  $g_{\text{new}} < g(m)$  then
11:       $g(m) \leftarrow g_{\text{new}}$ ;  $parent(m) \leftarrow n$ ;  $f(m) \leftarrow g(m) + h(m)$ 
12:      vlož/aktualizuj  $m$  v  $OPEN$  (a odeber z  $CLOSED$ , byl-li tam)
13:    end if
14:  end for
15: end while
16: return „cesta neexistuje“
```

Jaké záruky A* dává, závisí na vlastnostech heuristiky h . Rozlišují se u ní dvě, slabší a silnější.

Připustnost a konzistence heuristiky

Heuristika h je **připustná** (*admissible*), pokud nikdy nenadhadnocuje: $h(n) \leq h^*(n)$ pro všechna n , kde $h^*(n)$ je skutečná cena nejlevnější cesty z n do cíle.

Heuristika je **konzistentní** (*consistent*, monotónní), pokud pro každou hranu (n, m) platí trojúhelníková nerovnost

$$h(n) \leq w(n, m) + h(m), \quad h(g_{\text{cil}}) = 0.$$

Vlastnosti. Konzistence je silnější požadavek než připustnost, tedy konzistence $\Rightarrow$ připustnost, opačně to neplatí. Je-li heuristika připustná, je A^* *optimální*, tedy najde nejlevnější cestu. Konzistentní heuristika navíc zaručí, že f je podél každé cesty neklesající a že se každý vrchol expanduje *nejvýše jednou*.

Speciální případy a varianty. Položíme-li $h \equiv 0$, degeneruje A^* na **Dijkstrův algoritmus**. Na mřížce se typicky používá heuristika manhattanská $|\Delta x| + |\Delta y|$ pro 4 směry, oktilová pro 8 směrů a eukleidovská. **Weighted A^*** počítá $f = g + \varepsilon h$ s $\varepsilon > 1$: doběhne rychleji, ale nalezená cesta může být až $\varepsilon \times$ dražší než optimum. **IDA*** zase šetří paměť, protože si vystačí s $O(d)$ místo exponenciálního nároku.

Zařazení mezi prohledávací algoritmy. A^* je jeden bod na škále, na jejíchž koncích stojí dvě krajnosti. *BFS* a *Dijkstra/uniform-cost* berou v úvahu jen dosavadní cenu g , takže jsou sice úplně a optimální, ale prohledávají všemi směry stejně. Naopak *greedy best-first* bere jen odhad h , což je rychlé, zato *neoptimální* a snadno se dá svést. A^* obojí kombinuje a při připustné h zůstává optimální. Prohledaný objem lze ještě řádově zmenšit obousměrným prohledáváním, které postupuje současně od startu a od cíle.

Zrychlení bez ztráty optimality. Na mřížkách se používá **JPS** (*jump point search*). Mřížka obsahuje obrovské množství stejně dlouhých cest, které se liší jen pořadím kroků, a JPS tuto symetrii odstraní: „nezájímavé“ vrcholy přeskakuje a expanduje jen *skokové body*. Jinou cestou jdou **Theta*** a další *any-angle* algoritmy, které uvolní požadavek, aby cesta vedla po hranách mřížky. Při expanzi zkusí napojit vrchol rovnou na *dědečka*, je-li mezi nimi přímá viditelnost, a dají tak podstatně kratší a přirozenější cesty. **HPA*** (*hierarchical*) rozdělí mapu na clustery a předpočítá cesty mezi jejich vstupy; hledá pak nejdřív hrubě mezi clustery a teprve potom uvnitř nich.

Malý příklad. Mějme mřížku 3×3 s pohybem do 4 směrů a cenou hrany 1, start v $(0, 0)$, cíl v $(2, 2)$ a překážku na $(1, 1)$. Manhattanská heuristika $h(x, y) = |2 - x| + |2 - y|$ je zde připustná i konzistentní. Start má $f = 0 + 4 = 4$, jeho nástupci $(1, 0)$ a $(0, 1)$ mají $g = 1$, $h = 3$, $f = 4$. Dále přijdou na řadu $(2, 0)$ a $(0, 2)$ s $g = 2$, $h = 2$, $f = 4$, pak $(2, 1)$ a $(1, 2)$ s $g = 3$, $h = 1$, $f = 4$ a konečně cíl $(2, 2)$ s $g = 4$, $h = 0$, $f = 4$. Všechny vrcholy na optimální cestě tedy mají $f = 4$, což je typický projev konzistentní heuristiky, která zbytek cesty odhaduje přesně.

3.3 Dynamické prostředí: D* Lite

Dosud jsme předpokládali, že mapa se během hledání nemění. Skutečný agent se ale pohybuje a průběžně zjišťuje, že skutečnost mapě neodpovídá: objeví se nová překážka, změní se cena průchodu. Plánovat A^* od nuly po každé takové zprávě je drahé. Řešením jsou **inkrementální** algoritmy, které při lokální změně přepočítají jen tu část výpočtu, které se změna opravdu dotkla.

LPA* a D* Lite

LPA* (*Lifelong Planning A**, Koenig, Likhachev, 2001) je inkrementální A^* pro pevný start a cíl. Ke každému vrcholu si kromě $g(n)$ udržuje ještě *lookahead* hodnotu

$$rhs(n) = \begin{cases} 0 & n = s, \\ \min_{p \in \text{pred}(n)} (g(p) + w(p, n)) & \text{jinak.} \end{cases}$$

Vrchol je **lokálně konzistentní**, pokud $g(n) = rhs(n)$. Změní-li se cena hrany, stanou se některé vrcholy nekonzistentními; ty se zařadí do prioritní fronty a změna se propaguje jen tam, kde je to potřeba.

D* Lite (Koenig, Likhachev, 2002) je LPA* prohledávající *od cíle ke startu*, aby spočtené hodnoty zůstaly použitelné i poté, co se robot posune; pohyb se dorovná korekcí prioritní fronty. Funkčně je ekvivalentní staršímu algoritmu **D*** (Stentz, 1994), je však výrazně jednodušší a stal se *de facto* standardem v mobilní robotice.

V neznámém prostředí se k tomu přidává *optimistický předpoklad (freespace assumption)*: neznámé buňky se považují za průchodné. Agent tedy vyrazí po naplánované cestě, a jakmile narazí, přeplánuje. To je přesně scénář, pro který byl D* navržen.

3.4 Hledání cest pro více agentů (MAPF)

♦ nad rámec slidů: není ve slidech, ale je to jediná vysloveně multiagentní část okruhu

Dosud cestu plánoval jediný agent a ostatní pro něj byli nanejvýš pohyblivé překážky. Jakmile se ale plánuje pro celou skupinu naráz, mění se zadání i jeho obtížnost.

Multi-Agent Path Finding, MAPF

Vstupem je graf $G = (V, E)$ a k agentů, kde agent i má start s_i a cíl g_i . Čas je diskrétní a v každém kroku každý agent buď přejde po hraně, nebo počká na místě.

Konflikty. Řešení musí být *bezkonfliktní*, a to ve dvou ohledech. *Vrcholový konflikt* nastane, když se dva agenti ocitnou ve stejném čase ve stejném vrcholu, *hranový, takzvaný „swap“ konflikt* tehdy, když si dva agenti v jednom kroku vymění pozici.

Účelová funkce se volí buď jako *sum-of-costs* $\sum_i cost_i$, nebo jako *makespan* $\max_i cost_i$. Nalezení *optimálního* řešení je NP-těžké.

Prioritizované plánování je oddělený přístup k úloze. Agenti se uspořádají podle priority a pro $i = 1, \dots, k$ se cesta agenta i hledá algoritmem A^* v *prostoru-čase*, kde stavem je dvojice ⟨vrchol, čas⟩, tak, aby se agent vyhnul už naplánovaným trajektoriím uloženým v **rezervační tabulce**. Tato varianta se nazývá *Cooperative A** (Silver, 2005). Je rychlá a dobře škáluje, zaplatí se to však **neúplností**: existují řešitelné instance, které nevyřeší žádné pořadí priorit. Typickým příkladem jsou dva agenti v úzkém koridoru, kde jeden musí uhnout do výklenku.

Úplnost i optimalitu vrací zpět jiný přístup. Nesnaží se konfliktům předejít pevným pořadím agentů, ale řeší je až ve chvíli, kdy se opravdu objeví.

Conflict-Based Search (CBS)

Sharon a kol., 2012–2015. Dvouúrovňový *optimální a úplný* algoritmus, který se vyhýbá exponenciálnímu sdruženému stavovému prostoru V^k .

Nízká úroveň hledá pomocí A^* optimální cestu jednoho agenta v *prostoru-čase*, a to při respektování množiny *omezení* tvaru $\langle a_i, v, t \rangle$ („agent i nesmí být ve vrcholu v v čase t “).

Vysoká úroveň prohledává *strom omezení* podle ceny. Kořen má prázdnou množinu omezení a individuálně optimální cesty. Vezme se uzel s nejnižší cenou; jsou-li jeho cesty bezkonfliktní, je řešení optimální. Jinak se vybere první konflikt a uzel se *rozvětví* na dva potomky, přičemž do každého z nich se přidá omezení pro jednoho z kolidujících agentů a přeplánuje se *jen on*.

Škálovatelné varianty. Optimální CBS zvládne desítky agentů. Jde-li o stovky až tisíce, jak je tomu ve skladech Amazon/Kiva, obětuje se optimalita kontrolovaným způsobem. *ECBS* je *bounded suboptimal*: na obou úrovních používá *focal search* a jeho výsledek je zaručeně nejvýše $w \times$ horší než optimum. *SIPP* (*safe interval path planning*) nepracuje s jednotlivými časovými kroky, ale se *souvislými intervaly*, v nichž je vrchol volný, takže velikost stavového prostoru nezávisí na délce

horizontu. *MAPF-LNS* (*large neighborhood search*) opakovaně zahodí a znovu naplánuje cesty malé podmnožiny agentů.

3.5 Shrnutí ke zkoušce

- Algoritmus A^* pracuje s $f = g + h$ a jeho záruky plynou z vlastností heuristiky: *přípustnost* $h \leq h^* \Rightarrow$ optimalita, *konzistence* $h(n) \leq w(n, m) + h(m) \Rightarrow$ každý vrchol se expanduje jen jednou. Pro $h \equiv 0$ dostaneme Dijkstrův algoritmus, weighted A^* je ε -suboptimální.
- V dynamickém prostředí se plánuje inkrementálně: LPA^* si vede hodnoty g a rhs a sleduje lokální konzistenci $g = rhs$, D^* Lite prohledává od cíle a při změně opravuje jen zasaženou část. V neznámém prostředí se k tomu přidává *freespace assumption*.
- V úloze MAPF se rozlišují konflikty vrcholové a hranové (swap) a účelová funkce se volí jako sum-of-costs, nebo jako makespan. Optimální MAPF je NP-těžký.
- Prioritizované plánování, tedy Cooperative A^* s rezervační tabulkou, je rychlé, ale neúplné. **CBS** má nízkou úroveň v podobě A^* s omezeními a vysokou úroveň, která větví strom omezení podle nalezeného konfliktu; je optimální a úplný. Škálovat lze pomocí ECBS, SIPP a LNS.
- O kvalitě cesty rozhoduje diskretizace prostoru víc než algoritmus. Na výběr je mřížka, graf viditelnosti (optimální v rovině s mnohoúhelníky), navmesh, stavová mřížka, PRM a RRT pro vysoké dimenze a potenciálová pole, která jsou levná, ale uvíznou v lokálním minimu.
- Co se zařazení týče, BFS a Dijkstra berou jen g , greedy best-first jen h a A^* obojí. Bez ztráty optimality zrychlují JPS (odstraní symetrii v mřížce), Theta* (any-angle) a HPA^* (hierarchie).

4 Komunikace a znalosti v MAS: ontologie, řečové akty, FIPA-ACL, protokoly

4.1 Od agenta k multiagentnímu systému

Navrhnout jednoho agenta už umíme. Aby ale z několika agentů vznikl systém, musí spolu komunikovat, a to je víc než jen protáhnout mezi nimi drát. Vyřešit se musí pět věcí najednou:

- **Reprezentace znalostí**, tedy v čem vůbec jsou znalosti vyjádřeny.
- **Společný slovník**, aby stejné slovo znamenalo u obou stran totéž. Přesně to je úloha *ontologií*.
- **Standardní zprávy**, které komunikaci dávají pevnou syntaxi a sémantiku (*ACL*).
- **Předvídatelné chování při komunikaci** — konverzace se vede podle *interakčních protokolů*, takže je dopředu zřejmé, co může přijít dál.
- **Technické problémy**, které zůstanou i poté: systém je distribuovaný, agenti se mohou pohybovat, běží asynchronně a komunikační kanály nejsou spolehlivé.

První tři body jsou obsahem této kapitoly a začínají u slovníku, protože bez sdílených pojmů nemá smysl řešit ani formát zpráv, ani jejich pořadí.

4.2 Co je ontologie

Slovo ontologie má dva různě staré významy a oba se hodí znát, protože informatický pojem si z toho filozofického půjčuje ambici popsat, co ve světě existuje.

Ontologie

Filozoficky je ontologie nauka o bytí. Jméno vzniklo z řeckého *to on* (jsoucí) a *logos* (slovo); Aristotelés ji nazval „první filozofií“ a řadil ji do metafyziky.

V *informatice* znamená ontologie **explicitní a formalizovaný popis části reality**. Obvykle má podobu formálního deklarativního popisu, který obsahuje *glosář* (definice konceptů) a *tezaurus* (definice vztahů mezi koncepty). Ontologie je jakýsi slovník pro ukládání a výměnu znalostí o doméně standardním způsobem.

Gruber (1995): „Ontologie je popis (podobný formální specifikaci programu) konceptů a vztahů, které mohou pro agenta nebo komunitu agentů formálně existovat.“ Často citovaná zkrácená verze: *ontologie je explicitní specifikace konceptualizace*.

Motivační příklad (dialog dvou lidí, nebo stejně tak dvou agentů). „Slyšel jsi 7777? — Ne, co to je? — To je nové CD kapely SRPR, takový alt.rock s elektronikou a skvělými texty. — Aha.“ Aby si oba rozuměli, musí sdílet celou malou teorii hudebního světa. 7777 je album a spadá do žánrů alt.rock a electronica. Album má skladby, interpreta (zde SRPR) a autory hudby a textů. SRPR je kapela, kapela má členy a ti hrají na nástroje. . .

Aby se takový popis dal zapsat strojově, potřebuje pevnou kostru několika stavebních prvků.

Formalismus ontologie

- **Třídy** (*classes*, koncepty) sdružují věci, které mají něco společného.
- **Instance** (*individuals*, objekty) jsou konkrétní jedinci patřící do tříd.
- **Vlastnosti** (*properties*) popisují atributy tříd a instancí.
- **Relace mezi třídami** spojují třídy navzájem; nejdůležitější z nich je **podtřída** (*is-a*), která je *tranzitivní*.
- Zbytek tvoří další relace a vlastnosti, tedy základní znalosti o doméně.

Strukturálním částem (třídám, relacím a axiomům) se obvykle říká vlastní *ontologie*; teprve spolu s **fakty o konkrétních jedincích** tvoří **znalostní bázi** (*knowledge base*).

Ontologie ontologií podle síly formalizace. Ontologie se navzájem liší tím, jak přísně jsou formalizované, a mezi nejslabší a nejsilnější podobou vede plynulá škála (od slabých k silným): slovník (řízený slovník vybraných termínů) → glosář (definice významů v přirozeném jazyce) → tezaurus (synonyma) → neformální hierarchie (Amazon, wiki) → formální *is-a* hierarchie (subsumpce tříd) → třídy s vlastnostmi → omezení hodnot („každý člověk má právě jednu matku“) → libovolná logická omezení. Čím dál po škále postoupíme, tím víc se dá vyjádřit a odvodit, ale tím draž: poslední stupeň už vede na složité odvozovací algoritmy.

Ontologie ontologií podle rozsahu. Druhé dělení se řídí tím, jak široký výsek světa ontologie popisuje:

- **Aplikační** ontologie jsou v praxi nejběžnější, zato se těžko používají znovu jinde.
- **Doménové** ontologie popisují celý obor a jsou oblíbeným předmětem výzkumu sémantického webu.
- **Horní** ontologie (*upper ontologies*) by měly popisovat „všechno“ (Thing, Living thing, . . .) a sloužit jako základ doménových ontologií. Patří sem BFO, GFO a UFO; WordNet a IDEAS naopak nejsou pro formální odvozování vhodné. Proti samotné snaze vybudovat univerzální ontologii ostatně stojí metodologické námitky (Wittgenstein, *Tractatus logico-philosophicus*).

4.3 Ontologické jazyky

Ontologii je potřeba v něčem zapsat. K tomu slouží ontologické jazyky: formální deklarativní jazyky pro reprezentaci znalostí, které obsahují fakta i odvozovací pravidla. Nejčastěji se opírají o logiku prvního řádu nebo o *deskripční logiku*.

Historickým předchůdcem těchto jazyků jsou **rámce** (*frames*) od Minského, oblíbené v klasické UI a v expertních systémech. Jejich stavba téměř přesně odpovídá objektovému programování:

Terminologie rámců	Objektová terminologie
Frame (rámec)	třída objektů
Slot	vlastnost / atribut objektu
Trigger, accessor, mutator	metoda

XML pro ontologie vytvořen nebyl, přesto se k nim používá. Jeho hlavní výhodou je možnost definovat vlastní značky, které samy o sobě přirozeně reprezentují slovník domény:

```
<product typ="CD">
  <title>7777</title>
  <artist>SRPR</artist>
</product>
```

Chybí mu ovšem definovaná sémantika: popisuje strukturu dokumentu, nikoli význam toho, co v dokumentu stojí.

RDF — Resource Description Framework

Standardní nástroj pro reprezentaci znalostí (nejen) pro web. Je záměrně **jednoduchý**, tedy málo expresivní, zato s rychlými odvozovacími algoritmy.

Vše se vyjadřuje **trojicemi** *subjekt – predikát – objekt*, tedy *Predikt(Subjekt, Objekt)*. Například *MarriedTo(Karel, Jja)*, *FatherOf(Karel, Pa)*. Množina trojic tvoří orientovaný ohodnocený graf (*RDF graph*) a zdroje se v něm identifikují pomocí IRI. Serializovat se dá do RDF/XML, Turtle, N-Triples nebo JSON-LD, dotazuje se jazykem SPARQL. Nadstavba **RDFS** přidává třídy a konstrukce `subClassOf`, `domain` a `range`.

OWL — Web Ontology Language

Rovněž pochází ze (sémantického) webu a existuje ve verzích 1 a 2. Není to jeden jazyk, ale *kolekce několika formalismů* pro popis ontologií; zapsat se dají v XML, v RDF, funkcionální notaci, Manchester syntaxí nebo v Turtle. Za vším stojí snaha o jazyk, který je formální, a přesto prakticky použitelný.

Základem je **deskripční logika**, tedy *rozhodnutelný fragment* logiky prvního řádu.

Platí **předpoklad otevřeného světa** (*open world assumption*, OWA): co nevíme, je *nedefinované*. Databáze a Prolog naopak stojí na předpokladu uzavřeného světa, kde je vše, co nevíme, *nepravdivé*.

Deskripční logika

Pracuje se třemi druhy entit: **koncepty** (třídy), **role** (vlastnosti, predikáty) a **individa** (objekty). **Axiom** je logický výraz o rolích a konceptech. Zatímco rámce a objektové programování popisují třídu *úplně*, axiomy ji popisují jen *částečně* a mohou navíc přicházet z více zdrojů najednou.

Nejde o jeden systém, ale o celou rodinu logických systémů podle toho, jaké konstruktory se povolí. Základní **ALC** nabízí negaci, konjunkci a disjunkci konceptů a omezené existenční a univerzální kvantifikátory. Rozšíření se značí písmeny: hierarchie rolí (**H**), tranzitivita (**S**), inverzní role (**I**), kardinalitní omezení (**N**, **Q**), nominály (**O**).

Znalostní báze se dělí na dvě části: **T-Box** je terminologie, tedy axiomy o konceptech a rolích, a **A-Box** je asertivní část, tedy fakta o individech.

Profily OWL. Verze 1 se dodává ve třech odstupňovaných profilech. *OWL 1 Lite (SHIF)* je nejjednodušší a stojí blízko RDF; má mnoho omezení, aby šel rychle strojově zpracovat. *OWL 1 DL (SHOIN)* odpovídá deskripční logice, umožňuje tedy vyjádřit například disjunktnost tříd, a přitom zůstává úplný a rozhodnutelný. *OWL 1 Full* má největší vyjadřovací sílu, jenže mnoho problémů je v něm nerozhodnutelných. *OWL 2 (SROIQ)* k tomu přidává tři profily šité na míru typickým úlohám: *EL* odvozuje v polynomiálním čase a používají ho velké lékařské ontologie typu SNOMED, *QL* cílí na dotazy nad znalostní bází a napojení na relační databáze a *RL* zapisuje axiomy ve tvaru pravidel.

KIF — Knowledge Interchange Format

Reprezentuje znalosti v logice prvního řádu, a to LISPovskou prefixovou syntaxí. Navržen byl v rámci projektu DARPA *Knowledge Sharing Effort* jako **vnitřní jazyk** zpráv, tedy jazyk jejich obsahu, zatímco KQML byl jazykem *vnějším*, tedy obálkou. Příklady:

```
(salary 015-46-3946 widgets 72000)
(> (* (width chip1) (length chip1)) (* (width chip2) (length chip2)))
(forall (?F ?T) (=> (and (instance ?F Farmer) (instance ?T Tractor)) (likes
?F ?T)))
(exists (?F ?T) (and (instance ?F Farmer) (instance ?T Tractor) (likes ?F
?T)))
```

Přímým předchůdcem OWL byl ještě **DAML+OIL**, tedy spojení DARPA Agent Markup Language a Ontology Interchange Language; opuštěn byl v roce 2006.

4.4 Uvažování o znalostech: epistemická logika

♦ nad rámec slidů: slidy modální logiky výslovně vynechávají; okruh ale mluví o „znalostech v MAS“, takže minimum se hodí umět]

Ontologie říká, co znalost znamená. Epistemická logika řeší něco jiného: jak o znalosti mluvit zvenčí, tedy jak zapsat, že jeden agent něco ví, a jak z toho vyvozovat důsledky pro celou skupinu.

Epistemická logika a společná znalost

Znalost agenta se modeluje modálním operátorem $K_i\varphi$ („agent i ví, že φ “) s **kripkovskou sémantikou možných světů**: $K_i\varphi$ platí ve světě w , právě když φ platí ve *všech* světech, které agent i neumí od w odlišit. **Znalost** se axiomatizuje systémem **S5** a klíčový je v něm axiom **T**: $K_i\varphi \Rightarrow \varphi$, tedy co vím, to platí. **Přesvědčení** (operátor B_i , srov. BDI) se naproti tomu axiomatizuje systémem **KD45**, kde axiom T nahrazuje slabší **D**: $\neg B_i \perp$. Přesvědčení tedy musí být konzistentní, ale *může být mylné*. Formálně je rozdíl mezi znalostí a přesvědčením právě axiom T.

Pro skupinu G se zavádí $E_G\varphi$, tedy že každý v G ví φ . Silnější je **společná znalost** $C_G\varphi \equiv E_G\varphi \wedge E_GE_G\varphi \wedge \dots$, tedy že všichni vědí, že všichni vědí, ...

Koordinovaný útok (problém dvou generálů) ukazuje, proč na tom záleží. Dva generálové se přes nespolehlivého posla domlouvají na současném útoku, jenže žádný konečný počet potvrzení nevytvoří společnou znalost: po k zprávách platí nejvýše $E_G^k\varphi$, nikdy $C_G\varphi$. Poučení pro MAS je nepříjemné a obecné, totiž že *nespolehlivá komunikace nikdy nevytvoří společnou znalost*, a tedy ani dokonalou koordinaci.

4.5 Proč je komunikace agentů jiná

V objektovém programování znamená „komunikace“ volání metody objektu s parametry a volaný objekt *musí* vyhovět. U agentů to takhle nefunguje: nikdo nemůže jiného agenta přinutit, aby něco udělal, ani mu nesmí přímo přepsat vnitřní proměnné. Zbývá jediná cesta, totiž **provést komunikační akt** s jedním ze dvou cílů:

- vyměnit si s ostatními informace, nebo
- *ovlivnit* jiné agenty, aby něco udělali.

Ostatní agenti mají vlastní cíle a je jen na nich, jak s přijatou informací, žádostí či dotazem naloží. To je zásadní rozdíl: **komunikace v MAS je akce, ne volání**.

4.6 Teorie řečových aktů

Pokud je komunikace akce, potřebujeme teorii, která to řekne přesně. Multiagentní systémy si ji vypůjčily z filozofie jazyka.

Řečové akty (*speech acts*) — Austin, 1962

Některé části užívání jazyka mají charakter **akcí**, protože mění stav světa podobně jako akce fyzické. Právě těm se říká řečové akty:

- „Prohlašuji vás mužem a ženou.“
- „Vyhlašuji válku.“

Je to *pragmatická* teorie jazyka: zajímá ji, jak se řeč používá k dosahování cílů. Typickými performativními slovesy jsou *request*, *inform* a *promise*.

Jeden a týž výrok se přitom dá popsat na třech úrovních a pro agenty je použitelná jen jedna z nich.

Tři vrstvy řečového aktu

- **Lokuční akt** (*locutionary act*) zachycuje, *co bylo řečeno*, tedy samotný výrok. „Udělej mi čaj.“
- **Ilokuční akt** (*illocutionary act*) zachycuje, *co bylo míněno*, tedy lokuci plus performativní význam (dotaz, žádost, ...). „Požádal mě o čaj.“
- **Perlokuční akt** (*perlocutionary act*) zachycuje, *co se skutečně stalo*, tedy efekt řečového aktu. „Přiměla mě, abych jí uvařil čaj.“

Pro MAS je klíčová vrstva **ilokuční** — ta se stává obsahem zprávy (performativa).

Austin popsal, co řečové akty jsou. **Searle** k tomu rozpracoval podmínky, za nichž je řečový akt „zdařilý“. Pro akt *MLUVČÍ požádá POSLUCHAČE o akci* vypadají takto:

- **Standardní vstupně-výstupní podmínky** zajišťují, že komunikace vůbec proběhla: posluchač slyší, nejde o scénu ve filmu. ...
- **Předpoklady** shrnují, co musí platit, aby mluvčí mohl tento akt zvolit. Posluchač musí být schopen akci provést, mluvčí tomu musí věřit a nesmí být zřejmé, že by ji posluchač provedl i bez požádání.
- **Upřímnost** (*sincerity*) znamená, že mluvčí opravdu chce, aby se akce provedla.

Searle rovněž setřídil řečové akty samotné.

Searlovy kategorie řečových aktů

Starší dělení bylo jen výčtem druhů: *request*, *advice*, *statement*, *promise*, ...

Novější, a dnes standardní, dělení rozlišuje pět tříd:

- **Asertiva** (*assertives*, reprezentativa) informují posluchače (*inform*).
- **Direktiva** (*directives*) žádají posluchače o akci (*request*).
- **Komisiva** (*commissives*) zavazují mluvčího k něčemu do budoucna (*promise*).
- **Expresiva** (*expressives*) vyjadřují mentální stav či emoci mluvčího („děkuji“).
- **Deklarace** (*declarations*) samy mění stav věcí (válka, sňatek).

Každý řečový akt má vždy dvě složky: **performativní sloveso** (*request*, *query*, *inform*) a **pro-
poziční obsah** („okno je zavřené“).

Aby se s řečovými akty dalo počítat uvnitř agenta, musely dostat sémantiku srozumitelnou plánovači.

Plánovací teorie řečových aktů (Cohen, Perrault, 1979)

„...modelování [řečových aktů] v plánovacím systému jako operátorů definovaných... pomocí přesvědčení a cílů mluvčích a posluchačů. S řečovými akty se tak zachází stejně jako s fyzickými akcemi.“

Sémantika řečových aktů je tedy dána v duchu **STRIPS**, totiž trojicí předpoklady – delete – add. Rozdíl je jen v tom, že se v ní používají modální operátory pro *beliefs*, *abilities* a *wants*.

Příklad: Request a Inform.

	Request(S,H,A)	Inform(S,H, φ)
Předpoklad	$(S \text{ believe } (H \text{ cando } A)) \wedge$	$(S \text{ believe } \varphi)$
<i>cando</i>	$(S \text{ believe } (H \text{ believe } (H \text{ cando } A)))$	
Předpoklad <i>want</i>	$(S \text{ believe } (S \text{ want requestInstance}))$	$(S \text{ believe } (S \text{ want informInstance}))$
Efekt	$(H \text{ believe } (S \text{ believe } (S \text{ want } A)))$	$(H \text{ believe } (S \text{ believe } \varphi))$

Za pozornost stojí, že efektem *Inform není* „posluchač uvěří φ “, ale jen „posluchač uvěří, že mluvčí věří φ “. Autonomní agent totiž není povinen věřit tomu, co mu kdo řekne, a právě v tom se komunikační akt liší od volání metody.

4.7 KQML a FIPA-ACL

Teorie řečových aktů se do praxe promítla dvěma jazyky. Starší z nich je KQML.

KQML — *Knowledge Query and Manipulation Language*

Vznikl v 90. letech v rámci projektu DARPA *Knowledge Sharing Effort* (KSE) společně s KIF. Oba jazyky si rozdělily role:

- **KQML** je *vnější* komunikační jazyk, tedy „obálka dopisu“; nese *ilokuční* část zprávy.
- **KIF** je jazyk *vnitřní*: propoziční obsah zprávy a reprezentace znalostí.

Zpráva má podobu seznamu, v jehož čele stojí *performativum* a za ním následují pojmenované parametry:

```
(ask-one
  :content (PRICE IBM ?price)
  :receiver stock-server
  :language LPROLOG
  :ontology NYSE-TICKS)
```

Parametry: content, force, reply-with, in-reply-to, sender, receiver, language, ontology.

Performativa: achieve, advertise, ask-about, ask-one, ask-all, ask-if, break, sorry, error, broadcast, forward, recruit-one, recruit-all, reply, subscribe, tell, ...

Kritika KQML se soustředí do čtyř bodů. Chybělo performativum *commit*, tedy závazek. Sémantika nebyla přesně definovaná. Množina performativ byla nesystematická a zbytečně velká. A transportní mechanismus vůbec nebyl standardizován. Na tuto kritiku odpovídá druhý jazyk.

FIPA-ACL — *Agent Communication Language*

FIPA (*Foundation for Intelligent Physical Agents*, dnes standardizační komise IEEE) navrhla ACL jako **zjednodušení a zpřesnění KQML**: má menší a systematictější množinu performativ a *formálně definovanou sémantiku*. Praktickou implementací je platforma JADE.

Struktura zprávy čítá 13 polí, totiž performativum a 12 parametrů; povinné je přitom jediné, performative:

Pole	Význam
performative	typ komunikačního aktu (ilokuční síla) — <i>povinné</i>
sender, receiver, reply-to	identifikátory agentů (AID)
content	vlastní obsah zprávy
language	jazyk obsahu (SL, KIF, ...)
encoding, ontology	kódování a ontologie pro interpretaci obsahu
protocol	interakční protokol, jehož je zpráva součástí
conversation-id, reply-with, in-reply-to	párování zpráv do konverzace
reply-by	termín, do kdy se očekává odpověď

```
(inform
  :sender agent1
  :receiver agent2
  :content (price good2 150)
  :language sl)
```

:ontology hpl-auction)

Performativa FIPA-ACL. Je jich celkem 22 a přehledně se dělí podle toho, k čemu slouží; níže jsou ta nejdůležitější:

Skupina	Performativa
předávání informace	inform, inform-if, inform-ref, confirm, disconfirm
žádost o informaci	query-if, query-ref, subscribe
žádost o akci	request, request-when, request-whenever, cancel
vyjednávání	cfp (<i>call for proposals</i>), propose, accept-proposal, reject-proposal
provádění akcí	agree, refuse, failure
chyby, směrování	not-understood, proxy, propagate

♦ nad rámec slidů: formální sémantika SL ve slidech není, performativa ano]

Tím, čím se FIPA-ACL nejvíc odlišuje od KQML, je právě sémantika: každé performativum má předepsáno, kdy se smí použít a proč se používá.

Sémantika FIPA-ACL: FP a RE

Sémantika je definována v jazyce SL (*Semantic Language*), což je modální logika s operátory $B_i\varphi$ (agent i věří φ), $U_i\varphi$ (je nejistý), $I_i\varphi$ (má záměr), Feasible a Done. Ke každému performativu jsou dány dvě věci:

- **FP** (*feasibility precondition*) říká, co musí platit, aby odesílatel mohl akt provést;
- **RE** (*rational effect*) je efekt, kvůli němuž akt provádí. Příjemce *není povinen* RE naplnit, protože je autonomní — RE je jen důvod odesílatele.

Například pro $\langle i, \text{inform}(j, \varphi) \rangle$:

$$\text{FP: } B_i\varphi \wedge \neg B_i(Bif_j\varphi \vee Uif_j\varphi), \quad \text{RE: } B_j\varphi.$$

Čte se to takto: „já φ věřím a nevěřím, že už j o φ ví (že by věděl, zda platí, či neplatí — Bif — ani že by o tom měl mínění s nejistotou — Uif); chci, aby j uvěřil φ .“ Obdobně pro $\langle i, \text{request}(j, \alpha) \rangle$: FP: $FP(\alpha)[i \setminus j] \wedge B_i \text{Agent}(j, \alpha) \wedge \neg B_i I_j \text{Done}(\alpha)$, RE: $\text{Done}(\alpha)$.

Praktický problém je v tom, že tato sémantika je neverifikovatelná: zvenčí se nedá ověřit, že agent skutečně věří tomu, co tvrdí (*sincerity assumption*). Je to známá slabina mentalistické sémantiky ACL a alternativou k ní jsou *sociální* sémantiky založené na veřejných závazcích (*commitments*).

4.8 Interakční protokoly

Jednotlivá zpráva sama o sobě nestačí, protože agenti spolu vedou celé **konverzace** s předepsanou strukturou. Interakční protokol (*interaction protocol*) je vzor přípustných posloupností zpráv; FIPA jich standardizuje kolem tuctu ve specifikaci FIPA-IP a zapisuje je notací AUML.

Protokol	Průběh
FIPA-Request	$I \rightarrow P$: request. $P \rightarrow I$: refuse (konec), nebo agree; po vykonání inform-done / inform-result, případně failure. Odpověď může být i not-understood.
FIPA-Query	$I \rightarrow P$: query-if / query-ref. P : refuse nebo agree, dále inform s výsledkem nebo failure.
FIPA-Contract-Net	viz níže: cfp $\rightarrow n \times (\text{propose} / \text{refuse}) \rightarrow \text{accept-proposal}$ vítězi + reject-proposal ostatním $\rightarrow \text{inform-done} / \text{failure}$.
FIPA-Iterated-Contract-Net	jako CNET, ale iniciátor může po přijetí nabídek vyhlásit <i>další kolo</i> cfp (vyjednávání o lepších podmínkách).
FIPA-Subscribe	I : subscribe; P posílá inform pokaždé, když se sledovaná hodnota změní, dokud nepřijde cancel.
FIPA-Brokering Recruiting	/ I požádá zprostředkovatele (<i>broker</i>), ten najde vhodné poskytovatele, deleguje na ně požadavek a výsledky vrátí (příp. je nechá odpovědět přímo).
Aukční protokoly	FIPA-English-Auction, FIPA-Dutch-Auction (viz odst. 5.11).

Poznámka k návrhu. Protokol určuje, jaké zprávy jsou v daném stavu konverzace legální. Tím vnáší *předvídatelnost* do jinak autonomního chování a zároveň dovoluje implementovat agenta jako konečný automat nad stavy konverzace. Přesně to dělá JADE svými třídami typu *AchieveREInitiator* nebo *ContractNetResponder*.

4.9 Shrnutí ke zkoušce

- Ontologie je explicitní formalizovaný popis části reality, tedy glosář a tezaurus dohromady; standardně se cituje Gruberova definice. Formalismus tvoří třídy, instance, vlastnosti, relace (zejména tranzitivní *is-a*) a fakta, přičemž ontologie spolu s fakty dává znalostní bázi.
- Formalizace má škálu od pouhého slovníku až po libovolná logická omezení; podle rozsahu se ontologie dělí na aplikační, doménové a horní.
- RDF stojí na trojicích *subjekt-predikát-objekt*, je jednoduché a rychlé; RDFS k němu přidává třídy.
- OWL je postaveno na deskripční logice, tedy rozhodnutelném fragmentu FOL, platí v něm *open world assumption* a znalostní báze se dělí na T-Box a A-Box. Verze 1 má profily Lite, DL a Full (*SHIF*, *SHOIN*), OWL 2 je *SROIQ* a přidává profily EL, QL a RL.
- KIF je FOL v LISPovské syntaxi a tvoří obsah zprávy, KQML je obálka.
- Epistemická logika zapisuje znalost operátorem $K_i\varphi$ nad možnými světy; znalost se axiomatizuje systémem S5 s axiomem T, přesvědčení systémem KD45. Společná znalost $C_G\varphi$ nespolehlivou komunikaci nevznikne, což ukazuje koordinovaný útok.
- Komunikace agentů je *akce*, ne volání metody, takže příjemce není povinen vyhovět.
- Austin zavedl řečové akty a jejich tři vrstvy, lokuci, ilokuci a perlokuci. Searle k tomu přidal podmínky zdařilosti a pět kategorií (asertiva, direktiva, komisiva, expresiva, deklarace); každý akt se skládá z performativního slovesa a propozičního obsahu.
- Cohen a Perrault dali řečovým aktům sémantiku ve stylu STRIPS, tedy předpoklady a efekt, jen s modálními operátory. Request(S,H,A) a Inform(S,H, φ) patří k tomu, co se u zkoušky píše z paměti; efektem Inform je H věří, že S věří φ .
- KQML tvoří obálku a KIF obsah; k tomu patří parametry, performativa a kritika KQML (chybí commit, nejasná sémantika).
- FIPA-ACL má danou strukturu zprávy (performative, sender/receiver, content, language, ontology, protocol, conversation-id, reply-by), 22 performativ a sémantiku přes FP a RE v jazyce SL; její slabinou je neverifikovatelnost.
- Z protokolů se jmenují FIPA-Request, FIPA-Query, Contract Net i jeho iterovaná varianta, Subscribe, Brokering a aukční protokoly.

5 Distribuované řešení problémů, kooperace, teorie her a aukce

Nejobsáhlejší věta okruhu spojuje dvě situace, které spolu na první pohled souvisejí, ve skutečnosti však vedou k docela jiné matematice. Kapitola má proto dvě poloviny. V té první (odst. 5.1–5.6) agenti *sdílejí cíl* a zbývá vyřešit jedinou otázku, totiž jak si rozdělit práci; to je *distribuované řešení problémů* a *kooperace*. Ve druhé polovině (odst. 5.7–5.15) sleduje každý agent *vlastní zájmy*, a tak je potřeba teorie her s Nashovými ekvilibrii a Paretovou efektivitou, spolu s mechanismy odolnými vůči manipulaci, tedy alokací zdrojů a aukcemi.

5.1 Koherence a koordinace

Agenti mají různé cíle, jednají autonomně, pracují v čase a rozhodují se za běhu. Jejich součinnost proto nelze naplánovat dopředu, systém musí zvládnout *dynamickou koordinaci*. Sdílejí při ní dvě věci: **úlohy** (*task sharing*) a **informace** (*result sharing*).

Koherence a koordinace

Koherence (*coherence*) říká, jak dobře funguje systém *jako celek*: jakou kvalitu má nalezené řešení, jak efektivně se využívají zdroje a jak systém degraduje při poruchách.

Koordinace (*coordination*) měří něco jiného, totiž jak dobře se agentům daří *minimalizovat režii* spojenou se synchronizací. Dobře koordinovaný systém se vyhne zbytečně duplicitní práci, konfliktům o zdroje i zbytečné komunikaci.

Obojí spolu nesouvisí tak těsně, jak by se zdálo. Systém může být koherentní i při špatné koordinaci: agenti tutéž práci udělají třikrát, výsledek je ale správný. Zaplatí se to plýtváním.

CDPS — Cooperative Distributed Problem Solving

Pojem pochází od Lessera a kol. z 80. let. Označuje kooperaci jednotlivých agentů při řešení problému, který přesahuje jejich individuální schopnosti, ať už jde o informace, zdroje, nebo výpočetní kapacitu.

Celý přístup stojí na jednom předpokladu: agenti *implicitně sdílejí společný cíl* a konflikty mezi nimi nevznikají. Této vlastnosti se říká **benevolence**. Měřítkem úspěchu pak není prospěch jednotlivce, nýbrž *výkon systému jako celku*, takže agent celku pomůže i tehdy, když je to pro něj samotného nevýhodné. Benevolence *enormně zjednodušuje* návrh systému.

	Charakteristika	Rozdíl
PPS (<i>parallel problem solving</i>)	Bond, Gasser, 80. léta; důraz na paralelní řešení, homogenní a jednoduché procesory	agenti nejsou autonomní, jde v podstatě o paralelní algoritmus
CDPS	heterogenní agenti se sofistikovanými schopnostmi, sdílený cíl, benevolence	konflikty se neřeší, protože nevznikají
MAS	společnost agentů s <i>vlastními</i> cíli, které <i>nesdílejí</i>	je nutné řešit: proč a jak kooperovat, jak identifikovat a řešit konflikty, jak vyjednávat a smlouvat

5.2 Sdílení úloh a sdílení výsledků

Obecný postup CDPS probíhá ve třech fázích:

1. **Dekompozice problému** je hierarchická a rekurzivní. Ptáme se přitom na dvě věci: *jak* problém rozložit a *kdo* rozklad provede. Krajní variantou je model ACTORS, v němž pro každý podproblém vznikne nový agent, a to až na úroveň jednotlivých instrukcí.
2. **Řešení podproblémů** obvykle neprobíhá izolovaně: agenti si během něj typicky sdílejí informace a mohou potřebovat synchronizovat akce.
3. **Syntéza řešení** je opět hierarchická, skládají se při ní dílčí výsledky.

Obě sdílené věci fungují jinak. **Sdílení úloh** vyžaduje *dohodu* agentů o tom, kdo co udělá, a typickým mechanismem takové dohody je Contract Net. **Sdílení výsledků** naproti tomu může být buď *proaktivní*, kdy agent pošle výsledek, o němž si myslí, že se bude hodit, nebo *reaktivní*, kdy jej pošle až na vyžádání.

Přínosy sdílení výsledků

- **Jistota** (*confidence*): nezávislá řešení téhož problému lze porovnat, a shodnou-li se, důvěra ve výsledek roste.
- **Úplnost** (*completeness*): agenti sdílejí své *lokální* pohledy a tím vzniká globálnější představa o problému.
- **Přesnost** (*precision*): sdílení může zpřesnit celkové řešení.
- **Včasnost** (*timeliness*): zjevná výhoda distribuce, totiž zkrácení času.

5.3 Contract Net

Contract Net Protocol (CNET)

Smith a Davis, 1977–1980. Protokol staví na metafoře **sdílení úloh pomocí smluv** podle vzoru trhu: agent v nouzi vystupuje jako *manažer* (*manager*), ostatní jako *dodavatelé* (*contractors*). Probíhá ve čtyřech fázích:

1. **Rozpoznání** (*recognition*): agent zjistí, že má problém, který sám nezvládne. Buď mu chybí schopnost, zdroje či informace, nebo by řešení dopadlo hůř, než kdyby úlohu předal.
2. **Ohlášení** (*announcement*): rozešle, typicky broadcastem, oznámení úlohy. To obsahuje *popis úlohy*, který může být i spustitelný, dále *omezení* na termín, cenu a kvalitu a konečně *meta-informace* o tom, kdo se smí hlásit.
3. **Nabídky** (*bidding*): příjemci zváží, zda mají zájem a schopnosti, a pošlou nabídku (*tender*) s cenou, časem a kvalitou.
4. **Přidělení a realizace** (*awarding, expediting*): manažer vybere vítěze a uzavře s ním smlouvu, vítěz úlohu vykoná a vrátí výsledek.

Vlastnosti: protokol je jednoduchý, decentralizovaný a robustní vůči výpadkům. Dodavatel se navíc může sám stát manažerem podúlohy, čímž vznikají *hierarchické kaskády subkontraktů*. Zátěž se tak rozděluje dynamicky. Jde o nejčastěji implementovaný koordinační mechanismus vůbec: FIPA jej standardizuje jako FIPA-Contract-Net a JADE jej nabízí jako hotové chování.

Limity: broadcast se při mnoha agentech prodrazí a celý protokol předpokládá benevolenci, tedy že dodavatel nelže o svých schopnostech. Jakmile tento předpoklad padne, je nutné sáhnout po aukčních mechanismech odolných vůči manipulaci (odst. 5.11).

5.4 Systémy s tabulí (blackboard)

Blackboard system (BBS)

Historicky vůbec první schéma pro kooperativní řešení problémů; použil je systém HEARSAY-II pro rozpoznávání řeči v 70. letech. Výsledky se sdílejí přes společnou datovou strukturu, které se říká **tabule** (*blackboard*).

Metafora je názorná. Kolem tabule sedí několik expertů a každý z nich na ni může psát i z ní číst. Na tabuli se dynamicky objevují úlohy, a jakmile některý expert uvidí úlohu, kterou umí řešit, zapíše na tabuli částečné řešení. Takto se pokračuje, dokud se na tabuli neobjeví řešení celé úlohy.

Role:

- **Tabule** je sdílená paměť, typicky strukturovaná do několika *úrovní abstrakce*, které odpovídají hierarchii cílů a akcí. Zásadní volbou je formalismus reprezentace informace.
- **Experti** (*knowledge sources*) jsou agenti řešící konkrétní typ úlohy. Reagují na cíle na tabuli, svůj cíl si z tabule přebírají a po vybrání vykonají akci. Každý z nich má seznam schopností

a svou efektivitu.

- **Arbitr** (*arbiter, control component*) vybírá, který expert smí k tabuli. Může být čistě reaktivní, nebo může uvažovat plány a maximalizovat očekávanou utilitu. Odpovídá za řešení problému na vyšší úrovni.

Nad tabulí je **nutné vzájemné vyloučení**, a právě tam vzniká úzké hrdlo systému.

Klady a zápory BBS:

- + Poskytuje jednoduchý mechanismus koordinace a kooperace, který pokrývá sdílení úloh i sdílení výsledků.
- + Experti o sobě *nemusí vědět* a přesto spolupracují (*loose coupling*); přidání dalšího experta nevyžaduje změnu ostatních.
- + Zprávy na tabuli lze přepisovat, a tím delegovat úlohy, vytvářet podúlohy nebo měnit experty.
- Všichni agenti musí mít přístup k tabuli a sdílet stejnou architekturu, a kolem tabule bývá „narávano“; částečně to řeší distribuované hašovací tabulky.

Příklad: BBWar. Tabule je zde hašovací tabulka, která mapuje požadované schopnosti na úlohy, a publikují se na ní „otevřené mise“. Experti tvoří hierarchii: agent *velitel* hledá úlohy typu „dobyj město“ (ATTACK-CITY) a převádí je na několik úloh „zaútoč na pozici“ (ATTACK-LOCATION). Vojáci různých typů si pak z tabule vybírají mise, na které mají schopnosti.

Příklad: FELINE (Wooldridge, Jennings, 1990) je distribuovaný expertní systém z doby před KQML a ACL. Zajišťuje sdílení znalostí a distribuci podúloh. Každý agent je pravidlový systém, který si udržuje dvě položky: *Skills*, tedy „umím dokázat/vyvrátit následující...“, a *Interests*, tedy „zajímá mě, zda následující platí...“. Komunikace má tvar trojice (odesílatel, příjemce, obsah), kde obsahem je hypotéza doplněná řečovým aktem (request, response, inform).

5.5 Distribuované omezující podmínky (DisCSP a DCOP)

♦ nad rámec slidů: není ve slidech; okruh ale mluví o „distribuovaném řešení problémů“, kam formální větve patří

DisCSP a DCOP

Distribuovaný CSP vzniká tak, že se proměnné $x_1, \dots, x_n$ s doménami D_i rozdělí mezi agenty, typicky po jedné proměnné na agenta, a omezení propojují proměnné *různých* agentů. Hledá se přiřazení splňující všechna omezení. Žádný agent přitom nevidí celý problém a komunikuje jen se sousedy v grafu omezení.

DCOP je optimalizační varianta téhož. Omezení jsou *měkká* a zadaná funkcemi ceny $f_{ij}(x_i, x_j)$; maximalizuje se $\sum_{ij} f_{ij}$, tedy sociální blahobyt.

Algoritmy: nejznámější je *ABT* (asynchronní backtracking, Yokoo). Agenti jsou v něm uspořádáni a posílají si dva druhy zpráv: *ok?* se svým přiřazením směrem níž a *nogood* zpět nahoru, jakmile nastane konflikt. Dále sem patří *ADOPT*, *DPOP* a *Max-Sum*. Typickými aplikacemi jsou rozvrhování schůzek, přidělování úkolů senzorům a alokace frekvencí.

5.6 Interakce agentů v prostředí

Agenti si mohou navzájem pomáhat i překážet, aniž by spolu vůbec *komunikovali*. Stačí, že ovlivňují prostředí:

- Roboti přenesou cihlu jen tehdy, když na ni tlačí ze dvou stran současně. To je pozitivní interakce a vyžaduje synchronizaci.
- Roboti se natlačí ke dveřím a pak je nemohou otevřít. To je negativní interakce a vyžaduje koordinaci.

Agenti mohou vytvářet společenství, hierarchie i nepřátelství. **Je nutné znát typy interakcí** v konkrétním MAS, protože bez této znalosti nelze navrhnout efektivní řídicí mechanismy. Nejjednodušším

modelovým případem, kterým se zabývá následující oddíl, je interakce dvou racionálních (sobeckých) agentů v prostředí připomínajícím hru.

5.7 Sobecký agent

Proč by měl být agent upřímný ohledně svých schopností? A proč by měl přidělenou úlohu vůbec dokončit? Je-li systém homogenní, tedy celý navržený námi, je benevolence dobrá strategie. Většinou ale systém obsahuje agenty s různými zájmy a mezi společným cílem a cíli jednotlivců vzniká konflikt. V řízení letového provozu chtějí bezpečnost všichni, jenže každá aerolinka chce přistát první. Někdy navíc ani není zřejmé, co je společný zájem. V takové situaci je lépe uvažovat **sobecké** (*self-interested*) agenty.

Sobecký agent

Množina možných výsledků $O = \{o_1, o_2, \dots\}$ je *společná* pro všechny agenty, preference už však nikoli: každý agent i má vlastní **utilitní funkci** $u_i : O \rightarrow \mathbb{R}$, která výsledky uspořádá.

Strategické myšlení pak zní: maximalizuj svou utilitu za předpokladu, že všichni ostatní jednají rovněž racionálně. Společnou utilitu tento postup *nemaximalizuje*, zato je *robustní*.

Poznámka: peníze nejsou pro člověka dobrou utilitou. Utilitní funkce peněz je nelineární a liší se pro bohaté a chudé.

5.8 Hra v normální formě

Hra dvou hráčů v normální formě

Máme agenty i a j s množinami akcí A_i, A_j . Výsledek není v moci ani jednoho z nich, protože jej určují obě volby najednou: $e : A_i \times A_j \rightarrow O$. Agent volí **strategii** s_i , a to ve dvou možných podobách:

- **ryzí** (*pure*) strategie je deterministická volba jedné akce;
- **smíšená** (*mixed*) strategie je rozdělení pravděpodobnosti nad ryzími strategiemi, tedy náhodný výběr.

Hru zapisujeme **výplatní maticí** (*payoff matrix*), v níž řádky odpovídají strategiím hráče i , sloupce strategiím hráče j a každá buňka obsahuje dvojici u_i/u_j .

Obecně je hra v normální formě trojice $\langle N, (A_i)_{i \in N}, (u_i)_{i \in N} \rangle$, kde N je množina hráčů.

Dominance

Strategie s_i **silně dominuje** strategii s'_i , pokud dává agentovi i *striktně* lepší výsledek proti *všem* strategiím protihráče. **Slabě dominuje**, pokud dává výsledek alespoň stejně dobrý proti všem a striktně lepší alespoň proti jedné; tak dominanci definuje i přednáška. Strategie je **dominantní**, pokud dominuje všem ostatním strategiím agenta i .

Rozdíl mezi oběma pojmy není akademický: pravdomluvnost ve Vickreyově aukci je jen *slabě* dominantní (odst. 5.11).

Racionální agent hraje dominantní strategii, existuje-li nějaká. Na tom stojí základní řešící technika, *iterované odstraňování dominovaných strategií* (IESDS): dominovanou strategii nikdy nikdo nezahraje, lze ji tedy z matice vyškrtnout, což může učinit dominovanými další strategie. Odstraňujeme-li *silně* dominované strategie, na pořadí nezáleží a výsledek je jednoznačný. U *slabě* dominovaných strategií na pořadí **záleží** a lze přijít o některá ekvilibria.

Nashovo ekvilibrium

Dvojice strategií (s_1^*, s_2^*) je v **Nashově ekvilibriu**, pokud platí obojí:

- hraje-li agent i strategii s_1^* , je pro agenta j nejlepší hrát s_2^* ;
- hraje-li agent j strategii s_2^* , je pro agenta i nejlepší hrát s_1^* .

Strategie s_1^* a s_2^* jsou tedy *vzájemně nejlepší odpovědi* (*mutual best responses*). Obecně pro n hráčů je profil $s^* = (s_1^*, \dots, s_n^*)$ Nashovým ekvilibriem, pokud pro každého hráče i a každou jeho

strategii s_i platí

$$u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*),$$

kde s_{-i} značí strategie všech ostatních. Jinými slovy: **žádný hráč nemá jednostranný důvod se odchýlit.**

Naivní hledání ekvilibrií pro n agentů a m strategií vyžaduje projít m^n profilů. Uvedená definice se týká *ryzích* strategií, a v nich ekvilibrium existovat nemusí: hra kámen–nůžky–papír žádné nemá, jiné hry jich naopak mají několik.

Věta 5.1 (Nashova věta, 1950). *Každá hra s konečným počtem hráčů a konečným počtem strategií má alespoň jedno Nashovo ekvilibrium ve smíšených strategiích.*

Výpočetní složitost. Hledání NE je *totální* vyhledávací problém: řešení vždy existuje a jde jen o to najít ho. Od roku 2006 víme, že tento problém je **PPAD-úplný**, velmi zjednodušeně řečeno „o něco méně beznadějně než NP-úplnost“. NP-úplnost pro problém, který má vždy řešení, ostatně ani nedává smysl. Pro tři a více hráčů to dokázali Daskalakis, Goldberg a Papadimitriou; těsně poté Chen a Deng doplnili i zbývající a nejtěžší případ *dvou* hráčů.

Paretova efektivita a sociální blahobyt

Profil strategií je **Pareto-optimální** (*Pareto efficient*), pokud neexistuje jiný profil, který by zlepšil výsledek jednoho agenta, aniž by zhoršil výsledek některého jiného. Řešení, které Pareto-optimální není, lze tedy zlepšit „zadarmo“.

Sociální blahobyt (*social welfare*) profilu je součet utilit všech agentů, $sw(o) = \sum_{i \in N} u_i(o)$. Maximalizovat sociální blahobyt dává smysl v kooperativních scénářích, tedy když agenti patří do jednoho týmu, mají jednoho vlastníka nebo řeší jednu úlohu; čím homogennější systém, tím lépe. Maximum sociálního blahobytu je vždy Pareto-optimální, obráceně to však neplatí.

5.9 Klasické hry

Věžňovo dilema (*Prisoner's dilemma*)

$i \backslash j$	C (spolupráce)	D (zrada)
C	3/3	0/4
D	4/0	1/1

Zkratka C znamená *cooperate*, tedy spolupracovat se spolupachatelem, zkratka D pak *defect*, tedy udat ho a dostat nižší trest. Výplaty mají kanonické značení: R (*reward* za oboustrannou spolupráci), P (*punishment* za oboustrannou zradu), T (*temptation*, tedy zradím spolupracujícího) a S (*sucker's payoff*, tedy jsem zrazen). Hra je věžňovým dilematem, právě když platí

$$T > R > P > S \quad \text{a často navíc} \quad 2R > T + S.$$

- $R > P$ zaručuje, že oboustranná spolupráce je lepší než oboustranná zrada.
- $T > R$ a $P > S$ znamenají, že *zrada je dominantní strategie pro oba agenty*.
- (D, D) je **jediné Nashovo ekvilibrium** hry.
- (D, D) je zároveň **jediné ne-Pareto-optimální** řešení hry.
- (C, C) maximalizuje sociální blahobyt ($3 + 3 = 6$).

Racionalita každého jednotlivce tedy vede ke kolektivně nejhoršímu výsledku.

Důsledky věžňova dilematu:

- **Tragédie obecní pastviny** (*tragedy of the commons*): sdílený zdroj, ať už pastvina, rybolov, nebo atmosféra, se vyčerpá, protože každý jednotlivec maximalizuje svůj užitek.
- Otevřená otázka, *co vlastně znamená být racionální* a zda jsou lidé racionální; experimentálně totiž lidé spolupracují častěji, než teorie předpovídá.
- **Stín budoucnosti** (*shadow of the future*): v *iterovaném* věžňově dilematu se celá situace mění.

Iterované vězňovo dilema a Axelrodovy turnaje

Hraje-li se PD *konečněkrát* a oba to vědí, vyjde zpětnou indukcí D v každém kole: v posledním kole není důvod spolupracovat, tedy ani v předposledním, a tak dál. Hraje-li se *nekonečně*, případně s neznámým koncem daným pravděpodobností pokračování, stává se spolupráce racionální. Přesně to říká *folk theorem*: každý výplatní profil lepší než minimax lze podepřít jako ekvilibrium opakované hry.

Robert Axelrod uspořádal v roce 1980 turnaje strategií pro iterované PD. Vyhrála, a to dvakrát, nejjednodušší přihlášená strategie **Tit For Tat** (TFT, „půjčka za oplátku“): *v prvním kole spolupracuj, pak vždy zopakuj poslední tah soupeře*. Z výsledků odvodil Axelrod čtyři vlastnosti úspěšných strategií:

- **slušnost** (*nice*): nezrazuj jako první;
- **odvetnost** (*retaliating*): na zradu okamžitě odpověz;
- **odpouštění** (*forgiving*): vrátí-li se soupeř ke spolupráci, odpusť mu;
- **nezávistivost** (*non-envious*): neusiluj o to porazit *konkrétního* soupeře. TFT nikdy nezískal víc bodů než jeho protihráč, a přesto vyhrál turnaj.

Jako pátou vlastnost Axelrod uvádí **srozumitelnost** (*clarity*): strategie musí být pro protihráče čitelná, jinak s ní nelze navázat spolupráci. TFT má i slabinu. Objeví-li se šum, tedy chybně provedený tah, zaseknou se dva hráči TFT ve vzájemné odvetě; proto vznikly varianty *Generous TFT*, která občas odpustí, a *Tit for Two Tats*.

Koordinální hra

$i \backslash j$	Balet	Zápas
Balet	1/2	0/0
Zápas	0/0	2/1

Hra je známá také jako *Battle of the Sexes* (BoS) nebo *Bach or Stravinsky*. Výplaty podporují *shodu* strategií, hráči se ale liší v tom, na které shodě jim záleží víc.

- Nashova ekvilibria v ryzech strategiích jsou dvě: (Balet, Balet) a (Zápas, Zápas).
- Sociální blahobyť i Paretovu optimalitu maximalizují opět tyto dvě dvojice.
- **Smíšené NE**: každý hráč hraje svou *preferovanější* možnost s pravděpodobností $2/3$ a méně preferovanou s $1/3$. Očekávaná utilita ve smíšeném ekvilibriu vychází jen $2/3$, tedy *méně* než v kterémkoli ryším ekvilibriu.

Výpočet smíšeného NE (obecný recept pro hry 2×2): hráč j volí pravděpodobnost q hraní Baletu tak, aby byl hráč i *indifferentní* mezi svými akcemi:

$$u_i(\text{Balet}) = 1 \cdot q + 0 \cdot (1 - q), \quad u_i(\text{Zápas}) = 0 \cdot q + 2 \cdot (1 - q).$$

Z rovnosti $q = 2 - 2q$ plyne $q = 2/3$, hráč j tedy hraje Balet, svou preferovanou volbu, s pravděpodobností $2/3$. Symetricky hráč i hraje Zápas s pravděpodobností $2/3$. Očekávaná utilita hráče i je pak $1 \cdot \frac{2}{3} = \frac{2}{3}$. **Klíčové pravidlo: ve smíšeném NE volí každý hráč pravděpodobnosti tak, aby byl protihráč indiferentní.**

Antikoordinální hra (*Chicken, Hawk–Dove*) je opakem té předchozí, protože výplaty naopak podporují volbu *různých* strategií: $u(\text{uhnu}, \text{uhnu}) = 5/5$, $u(\text{uhnu}, \text{nedám se}) = 1/6$, $u(\text{nedám se}, \text{uhnu}) = 6/1$, $u(\text{nedám se}, \text{nedám se}) = 0/0$. Ryzí NE jsou obě *asymetrické* dvojice, zatímco maximum sociálního blahobytu, tedy situace, kdy oba uhnou ($5 + 5$), NE *není*. Symetrické smíšené NE je $p = 1/2$ s očekávanou utilitou 3.

5.10 Sociální volba a volby

◆ nad rámec slidů: ve slidech je celá kapitola, oficiální znění okruhu ji ale nejmenuje — zařazeno jako doplněk k agregaci preferencí]

Teorie her řeší, jak se agenti zachovají při daných preferencích. *Teorie sociální volby* se ptá opačným

směrem: jak z individuálních preferencí *odvodit* kolektivní rozhodnutí. Slouží k tomu dva nástroje. **Funkce sociálního blahobytu** agreguje preference do úplného uspořádání $>_{sw}$, kdežto **funkce sociální volby** vybírá jednoho vítěze.

Volebních systémů existuje celá řada. *Většinový* žádá nad 50 % hlasů. *Prostá většina* (*plurality*) přiděluje bod za každé první místo. *Prostá většina s vyřazováním* (IRV) opakovaně vyřadí nejslabšího kandidáta a přerozdělí jeho hlasy. *Sekvenční většinové volby* staví pavouka párových soubojů, takže *pořadí zápasů ovlivňuje výsledek*. *Bordova metoda* dává $N - 1$ bodů za první místo až po 0 za poslední, a je tedy hlasováním konsenzuálním, nikoli většinovým. *Slaterův systém* hledá acyklické uspořádání minimalizující počet obrácených hran v turnajovém grafu, což je NP-těžké.

Bordovy kombinace opravují hlavní slabinu Bordovy metody, totiž to, že nemusí zvolit Condorcetova vítěze. Všechny tři ho zvolí, pokud existuje. *Blackova metoda* nejdřív zkusí Condorcetovu metodu, a není-li vítěz, vezme Bordova. *Baldwinova metoda* spočte Bordova skóre, vyřadí kandidáta s nejnižším, skóre *přepočte* jen mezi zbylými a celý postup opakuje. *Nansonova metoda* dělá totéž, jen v každém kole vyřadí všechny kandidáty pod průměrem, a konverguje proto rychleji.

Vítěz prosté většiny nemusí být většinový. Při rozdělení $o_1 = 40\%$, $o_2 = 30\%$, $o_3 = 30\%$ vyhrává o_1 , ačkoli ho 60 % voličů nechťelo. Odtud pramení **taktické hlasování**.

Condorcetův paradox

Existují situace, kdy *ať zvolíme kterýkoli výsledek, většina voličů s ním bude nespokojena*.

Příklad: $V_1 : A > B > C$, $V_2 : B > C > A$, $V_3 : C > A > B$. V párových soubojích A porazí B (2:1), B porazí C (2:1) a C porazí A (2:1). **Kolektivní preference je cyklická**, přestože všechny individuální preference jsou lineární, a *Condorcetův vítěz* tak neexistuje. Motiv je stejný jako u neexistence ryziho Nashova ekvilibria v kámen–nůžky–papír.

Kritéria volebních systémů

Paretova podmínka (jednomyslnost) žádá, aby platilo $X >_{sw} Y$, kdykoli preferují X před Y *všichni* voliči. *Splňuje* ji majority a Borda, *nesplňuje* ji sekvenční většinové volby.

Podmínka Condorcetova vítěze žádá, aby vyhrál ten, kdo porazí všechny v párovém srovnání. *Splňují* ji sekvenční většinové volby, *nesplňuje* plurality ani Borda.

Nezávislost na irelevantních alternativách (IIA) žádá, aby to, zda $X >_{sw} Y$, záviselo pouze na relativním pořadí X a Y u voličů. *Nesplňuje* ji prakticky žádný běžný systém.

Zbývají **neomezený definiční obor** (univerzalita) a **nediktátorství**.

Věta 5.2 (Arrowova věta o nemožnosti, 1951). *Pro tři a více kandidátů neexistuje preferenční volební systém, který by převedl individuální uspořádání na úplné a tranzitivní společenské uspořádání a zároveň splňoval neomezený definiční obor, nediktátorství, Paretovu podmínku a IIA.*

Jednodušeji řečeno: pro více než dva kandidáty je jedinou procedurou, která splňuje Paretovu podmínku a IIA, *diktatura*. Je to *negativní* výsledek o limitech kolektivního rozhodování. Neplatí z něj, že jediným funkčním systémem je diktatura, nýbrž že každý reálný systém musí některé kritérium obětovat, nejčastěji IIA.

Gibbardova–Satterthwaiteova věta jde ještě dál: každá surjektivní a nediktátorská funkce sociální volby pro tři a více kandidátů je **manipulovatelná**. Pro MAS je to zásadní zjištění. Agent je počítač a taktické hlasování mu nedělá morální problém, a proto se hledají mechanismy odolné vůči manipulaci, což je téma aukcí.

5.11 Aukce jako mechanismus alokace zdrojů

Zatímco sociální volba řeší, jak z individuálních preferencí složit jedno kolektivní rozhodnutí, aukce odpovídají na užší, zato velmi praktickou otázku: komu připadne vzácný zdroj a za kolik.

Aukce

Aukce je **tržní instituce**, v níž zprávy od obchodníků nesou cenovou informaci: buď nabídku ke koupi za danou cenu (*bid*), nebo nabídku k prodeji za danou cenu (*ask*). Instituce přitom dává přednost vyšším nabídkám ke koupi a nižším nabídkám k prodeji.

V MAS je aukce především **mechanismus, jak rozdělit vzácné zdroje mezi agenty**. *Reálně* takto funguje eBay i Sotheby's a takto se přidělují těžební povolení nebo frekvenční pásma pro mobilní operátory. V *informatice* se aukcí rozděluje procesorový čas, šířka pásma, výpočetní úlohy nebo reklamní pozice.

Aukce je **efektivní**, přidělí-li zdroj agentovi, který ho chce nejvíce, tedy tomu s nejvyšším oceněním. Zájmy obou stran jdou přitom proti sobě: prodávající chce cenu *maximalizovat*, kupující *minimalizovat*, a každý kupující se řídí vlastní utilitní funkcí.

Klasifikace aukcí

Aukce se třídí podle dvou nezávislých hledisek.

Podle ocenění předmětu se rozlišují tři situace:

- *soukromá hodnota* (*private value*) závisí jen na kupujícím samotném, jako u lístku na koncert;
- *společná hodnota* (*common value*) je pro všechny stejná, jenom ji nikdo předem nezná, jako u těžebních práv; odtud pochází *prokletí vítěze* (*winner's curse*), protože vyhrává ten, kdo se ve svém odhadu nejvíce spletl směrem nahoru;
- *korelovaná hodnota* obě předchozí možnosti kombinuje.

Podle protokolu se sleduje, zda vítěz platí první, nebo druhou cenu, zda se draží otevřeným vyvoláváním (*open cry*), nebo obálkovou metodou (*sealed bid*), a zda aukce proběhne v jediném kole, nebo ve více kolech.

5.12 Čtyři základní typy aukcí

Kombinacemi předchozích hledisek se v praxi ustálily čtyři klasické protokoly. Neliší se jen tím, jak dražba probíhá, ale hlavně tím, jaká strategie je pro kupujícího racionální.

Aukce	Protokol	Racionální strategie	Poznámky
Anglická	otevřené vyvolávání, rostoucí cena, platí se první (nejvyšší) cena	<i>dominantní</i> : přihazovat po ϵ , dokud cena nedosáhne mého ocenění, pak odejít	Nejběžnější (starožitnosti, umění, internet — asi 95 % aukcí). Typický případ, kdy kupující cenu přeplatí.
Holandská	otevřené vyvolávání, klesající cena, první, kdo přijme, platí aktuální cenu	<i>dominantní strategie neexistuje</i> ; čekat, až cena klesne těsně pod mé ocenění — ale je to hazard	Rychlá; zboží podléhající zkáze (květiny, ryby). Oblíbená u prodávajících. Neumožňuje odhadnout tržní cenu ani preference ostatních.
FPSB (<i>first-price sealed bid</i>)	jedno kolo, uzavřené obálky, vyhrává nejvyšší nabídka a platí svou cenu	<i>dominantní strategie neexistuje</i> ; nabídnout méně než své ocenění (<i>bid shading</i>) — optimum závisí na odhadu ostatních	Prodej nemovitostí, státních dluhopisů. Nenutí kupující použít svou utilitní funkci, neodhalí tržní cenu. <i>Strategicky ekvivalentní holandské aukci</i> .
Vickreyova (<i>second-price sealed bid</i>)	jedno kolo, uzavřené obálky, vyhrává nejvyšší nabídka, ale platí druhou nejvyšší	<i>dominantní</i> : nabídnout pravdivě své ocenění	Teoreticky nejoblíbenější; Google AdWords, filatelistické aukce. <i>Ekvivalentní anglické aukci</i> (se soukromými hodnotami).

Proč je pravdomluvnost dominantní strategií ve Vickreyově aukci

Nechť moje ocenění je v , nabídnu b a nejvyšší nabídka ostatních je p , kterou ovšem neznám. Vyhráji, právě když $b > p$, a pak platím p , takže můj zisk činí $v - p$. Zbývá projít obě možné odchylky od pravdivé nabídky.

- **Nadhodnotím, tedy nabídnu $b > v$.** Proti volbě $b = v$ se něco změní jedině tehdy, když $v < p < b$. Pak sice vyhráji, ale platím $p > v$, takže můj zisk $v - p < 0$ je záporný. *Nadhodnocení mi tedy může jen uškodit.*
- **Podhodnotím, tedy nabídnu $b < v$.** Proti volbě $b = v$ se něco změní jedině tehdy, když $b < p < v$. Pak prohrají, ačkoli bych byl vyhrál se ziskem $v - p > 0$. *O tento zisk se připravím.*
- V žádné situaci tedy *nemohu* nabídkou $b \neq v$ získat proti volbě $b = v$ víc.

Pravdivá nabídka $b = v$ je proto (slabě) dominantní strategie, a to bez ohledu na to, co dělají ostatní. Vickreyova aukce je tím pádem **odolná vůči manipulaci** (*strategy-proof, incentive compatible*) a zároveň **efektivní**, protože vyhraje agent s nejvyšším oceněním.

Příklad — srovnání výnosů. Do dražby jdou tři kupující s oceněními $A = 80$ Kč, $B = 60$ Kč a $C = 30$ Kč. V ideálním scénáři, tedy bez dodatečné informace o ostatních a bez podvádění, dopadnou jednotlivé aukce takto:

Aukce	Vítěz	Cena
Anglická	A	≈ 61 (přihazuje se, dokud neodstoupí B na 60)
Holandská	A	80 nebo o něco méně (79) — A nesmí riskovat, že přijme až pozdě
FPSB	A	80 (resp. $60 + \varepsilon$, zná-li A ocenění ostatních)
Vickreyova	A	60 (druhá nejvyšší nabídka)

Předmět ve všech čtyřech případech získá agent s nejvyšším oceněním, všechny aukce jsou tedy **efektivní**. Rozcházejí se až v ceně, kterou vítěz zaplatí, a v množství informace, kterou dražba cestou odhalí.

Proč u FPSB vychází 80, když se má nabízet pod svým oceněním? Protože scénář předpokládá *nulovou* informaci o ostatních. Agent A neví, jak vysoko půjde B , a tak pro něj každý ústup od 80 znamená riziko prohry. Míra podhodnocení je funkcí přesvědčení o ostatních, a právě proto FPSB ani holandská aukce **nemají dominantní strategii**.

Jakmile A ocenění ostatních zná, nabídne $60 + \varepsilon$ a výnos prodávajícího spadne na úroveň Vickreyovy aukce; srov. větu o ekvivalenci výnosů níže. U anglické a Vickreyovy aukce je situace jiná: dominantní strategie tam existuje a nezávisí na tom, co dělají ostatní.

Věta o ekvivalenci výnosů (*revenue equivalence theorem*)

Věta se pojí se jmény Vickrey (1961), Myerson a Riley–Samuelson. Předpokládá se, že kupující jsou rizikově neutrální, že jejich soukromé hodnoty jsou nezávislé a pocházejí ze stejného spojitého rozdělení, že předmět získá kupující s nejvyšším oceněním a že kupující s nejnižším možným oceněním má nulový očekávaný zisk. Za těchto předpokladů platí, že **všechny čtyři základní aukce dávají prodávajícímu stejný očekávaný výnos**.

Rozdíly, které mezi aukcemi pozorujeme v praxi, tedy pramení z porušení některého předpokladu. Jsou-li kupující averzní k riziku, vede si lépe FPSB a holandská aukce. Jde-li o společné hodnoty s hrozbou prokletí vítěze, je lepší anglická aukce, protože během vyvolávání odhaluje informaci. Hrozí-li naopak koluze mezi kupujícími, je anglická aukce zranitelnější. A tam, kde rozhodují náklady na komunikaci, vychází levněji obálková metoda.

5.13 Kombinatorické aukce a VCG

Dosud se dražil vždy jediný předmět. Jakmile jich jde do dražby několik najednou, mění se úloha podstatně, protože kupujícího zajímají spíš celé balíky předmětů než jednotlivé kusy.

Kombinatorická aukce

Draží se *více předmětů současně* a kupující podávají nabídky rovnou na *podmnožiny (bundles)*. Motivem je, že hodnoty předmětů nejsou aditivní. Někdy se předměty doplňují, což je *komplementarita*: levá a pravá bota nebo dva sousední frekvenční bloky mají dohromady vyšší hodnotu než zvlášť, tedy $v(\{x, y\}) > v(x) + v(y)$. Jindy se naopak zastupují, což je *substituce*: dvě letenky na tentýž let, tedy $v(\{x, y\}) < v(x) + v(y)$.

Aukcionář pak stojí před **problémem určení vítězů** (*winner determination problem*, WDP): má vybrat takovou množinu vzájemně disjunktních nabídek, která maximalizuje součet cen. Je to zobecnění množinového pakování, a tedy problém **NP-těžký**, navíc i těžko aproximovatelný. Řeší se ILP solvery a metodou větvení a mezí (CABOB, CPLEX).

Druhá potíž je *reprezentační*. Podmnožin, na které lze nabízet, je $2^m - 1$, a proto se nabídky zapisují v kompaktních jazycích (OR, XOR, OR-of-XOR).

5.14 Další typy aukcí a alokace zdrojů

Čtveřicí základních protokolů se repertoár aukcí nevyčerpává. Praxe si vynutila řadu dalších variant, které mění buď to, kdo nakonec platí, nebo to, kdo vůbec smí nabídku podat.

- V *all-pay auction* zaplatí svou nabídku všichni účastníci, ale předmět získá jen ten s nejvyšší nabídkou. Modeluje se tak lobbing, úplatky, sportovní klání i patentové závody.
- *Tichá* aukce je písemnou variantou anglické. *Amsterodamská* začíná jako anglická, a jakmile zbudou dva zájemci, přepne se na holandskou s dvojnásobnou cenou. Na *Tsukiji*, tokijském rybím trhu, nabídnou všichni najednou a remízy se rozhodují hrou kámen–nůžky–papír.
- V *dvojitě aukci* zadávají nabídky kupující i prodávající a protokol je páruje; tak funguje burza.

5.15 Návrh mechanismů, VCG a vyjednávání

◆ nad rámec slidů: ve slidech není; okruh jmenuje „alokaci zdrojů“, k níž toto patří — kombinatorické aukce slidy zmiňují jednou větou]

Návrh mechanismů (*mechanism design*)

Teorie her se ptá, *jak se agenti zachovají při daných pravidlech*. Návrh mechanismů otázku obrací, a proto se mu říká „obrácená teorie her“: ptá se, *jaká pravidla zvolit, aby racionální sobečtí agenti sami od sebe dělali to, co chceme*, tedy typicky aby mluvili pravdu.

Mechanismus je dvojice pravidel. První je pravidlo výběru výsledku $x(\hat{v})$ z ohlášených preferencí, druhé platební pravidlo $p_i(\hat{v})$; utilita agenta pak vychází jako $u_i = v_i(x(\hat{v})) - p_i(\hat{v})$. Od mechanismu se žádá několik vlastností najednou. **Kompatibilita s motivací** znamená, že se agentovi vyplatí ohlásit pravdu; v nejsilnější variantě *strategy-proof* je pravdomluvnost dominantní strategií. **Individuální racionalita** požaduje $u_i \geq 0$ a **efektivita** maximalizaci sociálního blahobytu; k tomu přistupuje **vyrovnaný rozpočet** a **výpočetní zvládnutelnost**. Vše najednou mít nelze.

Princip odhalení říká, že ke každému mechanismu existuje *přímý* mechanismus, v němž agenti ohlásí preference pravdivě a výsledek zůstane stejný. Proto se celá teorie točí kolem Vickreyho a VCG. Bez plateb ovšem naráží na Gibbardovu–Satterthwaiteovu větu.

Kombinatorické aukce a VCG

V **kombinatorické aukci** se draží více předmětů současně a nabídky se podávají na *podmnožiny*, protože hodnoty nejsou aditivní. Buď se předměty doplňují (*komplementarita*, $v(\{x, y\}) > v(x) + v(y)$, například dva sousední frekvenční bloky), nebo se zastupují (*substituce*, $v(\{x, y\}) < v(x) + v(y)$). **Problém určení vítězů** (WDP), tedy volba disjunktních nabídek maximalizujících součet cen, je **NP-těžký**.

VCG (Vickrey–Clarke–Groves) Vickreyovu aukci zobecňuje: zvolí se alokace x^* maximalizující $\sum_i \hat{v}_i(x)$ a agent i zaplatí **externalitu, kterou způsobuje ostatním**:

$$p_i = \max_x \sum_{j \neq i} \hat{v}_j(x) - \sum_{j \neq i} \hat{v}_j(x^*).$$

Mechanismus je pravdomluvný, efektivní i individuálně racionální a pro jediný předmět se redukuje na Vickreyovu aukci. Nevýhody jsou tři: WDP se musí řešit $n+1$ -krát, výnos prodávajícího bývá malý nebo i nulový a celý mechanismus je zranitelný vůči koluzi.

Příklad: draží se předměty $\{x, y\}$, přičemž agent 1 nabízí $\{x\} \rightarrow 5$, agent 2 nabízí $\{y\} \rightarrow 4$ a agent 3 nabízí $\{x, y\} \rightarrow 8$. Optimální alokace rozdělí předměty mezi první dva agenty, protože $5 + 4 = 9 > 8$. Agent 1 zaplatí $p_1 = 8 - 4 = 4$, neboť bez něj by ostatní měli 8, kdežto se zvolenou alokací mají 4; jeho zisk tedy činí $5 - 4 = 1$. Symetricky vychází $p_2 = 8 - 5 = 3$ a prodávající inkasuje jen $7 < 8$.

Vyjednávání: monotónní protokol ústupků a Zeuthenova strategie

Každá aukce vyžaduje centrálního aukcionáře. Alternativou, která se bez něj obejde, je přímé **vyjednávání** mezi agenty. *Vyjednávací množinu* tvoří ty dohody, které jsou individuálně racionální a zároveň Pareto-optimální.

V **monotónním protokolu ústupků** navrhnou v každém kole oba agenti dohodu současně. Vyjednávání skončí, je-li návrh jednoho z nich pro druhého alespoň tak dobrý jako jeho vlastní. Jinak musí alespoň jeden *ustoupit*; když neustoupí nikdo, vyjednávání ztroskotá.

Zeuthenova strategie odpovídá na otázku, *kdo* má ustoupit: ten, kdo má menší ochotu riskovat konflikt.

$$\text{Risk}_i = \frac{u_i(\text{vlastní návrh}) - u_i(\text{návrh protějšku})}{u_i(\text{vlastní návrh})}.$$

Ustoupí agent s *nižším* Risk, a to právě tak málo, aby se poměry obrátily. Dvojice Zeuthenových strategií je v Nashově ekvilibriu a výsledná dohoda maximalizuje součin utilit, čemuž se říká Nashovo vyjednávací řešení.

5.16 Shrnutí ke zkoušce

- Koherence hodnotí výkon celku, kdežto koordinace znamená minimalizaci synchronizační režie.
- CDPS stojí na benevolenci agentů a na sdíleném cíli; odlišit ho od PPS, kde jsou homogenní procesory a paralelní algoritmus, i od MAS, kde mají agenti vlastní cíle, vznikají konflikty a vyjednává se.
- CDPS probíhá ve třech fázích: dekompozice, řešení podproblémů, syntéza. Sdílení úloh vyžaduje dohodu, sdílení výsledků může být proaktivní i reaktivní a přináší čtyři přínosy: jistotu, úplnost, přesnost a včasnost.
- **CNET** má čtyři kroky: recognition, announcement, bidding a awarding/expediting. Rolemi jsou manažer a dodavatel, subkontrakty mohou být hierarchické a protokol je standardizován jako FIPA-Contract-Net.
- **BBS** tvoří tabule (hierarchie abstrakce, mutex), experti se svými dovednostmi (skills) a arbitr, který experta vybírá. Vazba mezi experty je volná a úzkým hrdlem je přístup k tabuli.
- V DisCSP a DCOP jsou proměnné a omezení rozdělené mezi agenty a nikdo nevidí celý problém; ABT si posílá zprávy *ok?* a *nogood*, DCOP navíc maximalizuje sociální blahobyt.
- Sobecký agent maximalizuje *svoji* očekávanou utilitu za předpokladu, že ostatní dělají totéž; společná utilita se tím nemaximalizuje.
- Ke hře v normální formě patří rozdíl mezi ryzími a smíšenými strategiemi, pojem dominance a dominantní strategie a iterované odstraňování dominovaných strategií.
- **Nashovo ekvilibrium** je profil vzájemně nejlepších odpovědí, z něhož se nikomu nevyplatí jednostranně odchytil. Naivní hledání projde m^n profilů, **Nashova věta** zaručuje existenci ve

smíšených strategiích a samotné hledání je PPAD-úplné.

- **Paretova optimalita** říká, že nelze zlepšit jednomu, aniž by se jinému přitížilo, zatímco **sociální blahobyt** je součet utilit; maximum sw je Pareto-optimální, obráceně to neplatí.
- **Věžňovo dilema** má výplaty $T > R > P > S$, strategie D je dominantní a (D, D) je jediné NE a zároveň jediný ne-Pareto-optimální profil, kdežto (C, C) maximalizuje sw. Stejnou strukturu má tragedy of the commons. U iterovaného PD se uplatní stín budoucnosti a Axelrodova TFT, která je slušná, odvetná, odpouštějící, nezávistivá a srozumitelná.
- Umět spočítat smíšené NE pro hru 2×2 : pravděpodobnosti se volí tak, aby byl indiferentní protihráč.
- Ze sociální volby znát plurality, IRV, sekvenční hlasování, Bordu a Slatery; Condorcetův paradox dává cyklickou kolektivní preferenci; kritéria jsou Pareto, Condorcetův vítěz, IIA, univerzalita a nediktátorství; **Arrowova věta** říká, že Pareto + IIA $\Rightarrow$ diktatura, a Gibbard–Satterthwaite, že každý systém lze manipulovat.
- Aukce je mechanismus alokace vzácných zdrojů a efektivní aukce dá zdroj tomu, kdo ho oceňuje nejvíc. Klasifikačními dimenzemi jsou private a common value, open cry a sealed bid, first a second price a počet kol.
- Čtyři základní typy se liší racionální strategií: anglická (přihazuj po ε do svého ocenění) a Vickreyova (nabídní pravdivě) *dominantní* strategii mají, kdežto holandská (čekej na ocenění $-\varepsilon$) a FPSB (nabídní pod svým oceněním) ji **nemají**. U Vickreyovy aukce umět **důkaz pravdomluvnosti**.
- Platí ekvivalence anglická $\leftrightarrow$ Vickreyova a holandská $\leftrightarrow$ FPSB. Znáť **větu o ekvivalenci výnosů** i případy, kdy neplatí (averze k riziku, common values, prokletí vítěze, koluze), a umět příklad s oceněními 80/60/30.
- Návrh mechanismů je obrácená teorie her a patří k němu princip odhalení; kombinatorické aukce vedou na WDP, který je NP-těžký, a **VCG** vybere alokaci maximalizující sw a účtuje platbu rovnou externalitě.
- Vyjednávání se vede monotónním protokolem ústupků doplněným Zeuthenovou strategií, podle níž ustupuje ten s nižším rizikem.

6 Metody pro učení agentů a zpětnovazební učení

[♦ nad rámec slidů: celá kapitola] Přednáška řadí učení agentů výslovně mezi témata, která *nepokrývá*. Okruh ho ale jmenuje, takže se bez něj u zkoušky obejít nedá. Kapitola je proto zpracovaná v rozsahu, který na zkoušce obtojí: přehled metod učení, k tomu MDP a Q-learning podrobně a zbytek jen rámcově.

6.1 Proč a jak se agenti učí

Od inteligentního agenta v MAS se čeká, že bude *adaptivní*, tedy že změní své chování podle toho, co zažil. Přesně to je učení: změna chování na základě zkušenosti. Cest k ní vede celá škála a liší se tím, odkud agent bere učební signál i tím, co se vlastně učí.

Metody pro učení agentů

- **Podle učebního signálu** se rozlišují tři klasická paradigmatu strojového učení. Při *učení s učitelem* dostává agent ke vjemům i správné odpovědi, ať už jde o učení modelu protivníka z pozorovaných tahů, nebo o klasifikaci vjemů. *Učení bez učitele* hledá strukturu v samotných pozorováních, typicky shlukuje situace nebo objevuje role. **Zpětnovazební učení** má k dispozici jen skalární odměnu z prostředí; pro autonomní agenty je nejpřirozenější a věnuje se mu zbytek kapitoly.
- **Imitační učení** (*learning from demonstration*) staví na trajektoriích, které předvedl expert. Agent je buď přímo kopíruje (*behavioral cloning*), nebo z expertova chování rekonstruuje odměnovou funkci, což je *inverzní RL*.

- **Evoluční metody** šlechtí genetickým algoritmem celou populaci kontrolérů, tedy parametrů reaktivních chování, vah neuronové sítě nebo pravidel, a vybírají podle dosažené utility. Je to typický způsob, jak se „experimentálně ladí“ reaktivní architektury z kap. 2; patří sem i *learning classifier systems* a neuroevoluce.
- **Podle toho, co se agent učí:** může to být model prostředí, tedy přechodová funkce, dále model protivníka (*opponent modelling*), hodnotová funkce, přímo politika, nebo u PRS ohodnocení plánů.
- **Individuální vs. sociální učení** se liší zdrojem zkušenosti. Buď agent těží jen z té vlastní, nebo i napodobuje ostatní a zkušenosti s nimi sdílí, jak to dělá učení v roji a sdílené zkušenosti v MARL.

Nejběžnější a teoreticky nejpropracovanější přístupem k učení v MAS je **zpětnovazební učení** (*reinforcement learning*, RL). Agent při něm mění chování metodou pokus–omyl podle *odměn*, které dostává z prostředí. Do formalismu z kapitoly 1 zapadá bez potíží: politika π není nic jiného než mechanismus výběru akce a odměna operacionalizuje utilitu.

Podstatné je, kolik agentů se učí zároveň, protože podle toho se rozlišují dva zcela odlišné případy:

- **Učení jednoho agenta** je snazší, protože si vystačí s klasickými algoritmy RL, především s Q-learningem.
- **Multiagentní RL (MARL)** řeší situaci, kdy se agenti učí *současně*. Formalizuje se markovskými hrami, u nichž se ptáme na Nashovu a Paretovu optimalitu, a state of the art je dnes hluboké MARL (MADRL).

6.2 Markovský rozhodovací proces

Aby se o odměnách a politikách dalo uvažovat přesně, potřebuje RL formální model prostředí. Tím modelem je markovský rozhodovací proces.

MDP — Markov Decision Process

MDP je pětice $\langle S, A, T, R, \gamma \rangle$, jejíž složky znamenají toto:

- S je prostor stavů a A prostor akcí;
- $T : S \times A \times S \rightarrow [0, 1]$ je přechodová funkce, $T(s, a, s') = P(s_{t+1} = s' \mid s_t = s, a_t = a)$;
- $R : S \times A \times S \rightarrow \mathbb{R}$ je funkce odměny, tedy okamžitá odměna za přechod;
- $0 \leq \gamma < 1$ je diskontní faktor a vyjadřuje kompromis mezi okamžitou a budoucí odměnou.

Platí **markovská vlastnost**: následující stav a odměna závisí pouze na aktuálním stavu a akci, nikoli na celé historii. Stojí za srovnání s přechodovou funkcí $\tau : R^A \rightarrow 2^E$ z abstraktní architektury, která historii *má*; MDP je její markovské zjednodušení obohacené o pravděpodobnosti a odměny.

Politika (*policy*) je zobrazení $\pi : S \rightarrow A$, případně stochasticky $\pi(a|s)$. Cílem je najít *optimální politiku* π^* , která maximalizuje očekávaný diskontovaný součet budoucích odměn:

$$\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \right].$$

Hodnotová funkce $V^\pi : S \rightarrow \mathbb{R}$ přiřazuje stavu očekávanou utilitu za předpokladu, že agent sleduje politiku π ; **akční hodnotová funkce** $Q^\pi(s, a)$ dělá totéž pro dvojici stav–akce.

Bellmanovy rovnice

Hodnotu stavu lze zapsat rekurzivně přes hodnoty jeho následníků. Pro danou politiku tak dostaneme *Bellmanovu rovnici pro evaluaci*:

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^\pi(s')].$$

Pro optimální politiku platí *Bellmanova rovnice optimality*:

$$V^*(s) = \max_{a \in A} \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')],$$

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a'} Q^*(s', a')].$$

Optimální politika se z Q^* získá jako $\pi^*(s) = \arg \max_a Q^*(s, a)$.

Iterace hodnot (*value iteration*) se použije tehdy, když známe úplný popis MDP. Bellmanova rovnice optimality slouží rovnou jako předpis pro přepočítání hodnot:

- 1: inicializuj $V_0(s) \leftarrow 0$ pro všechna s
- 2: **repeat**
- 3: **for all** $s \in S$ **do**
- 4: $V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$
- 5: **end for**
- 6: **until** $\max_s |V_{k+1}(s) - V_k(s)| < \varepsilon$
- 7: **return** $\pi(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V(s')]$

Že tento postup skutečně někam dojde, plyne z toho, že Bellmanův operátor je kontrakce s koeficientem γ ; iterace proto konverguje ke V^* geometricky rychle. Alternativou je **iterace politiky** (*policy iteration*), která místo hodnot vylepšuje rovnou politiku a střídá přitom dva kroky: nejprve aktuální politiku *vyhodnotí* vyřešením soustavy lineárních rovnic, pak ji hladově podle V^π *zlepší*. Konverguje v konečně mnoha krocích, protože politik je konečně mnoho.

6.3 Q-learning a SARSA

Předchozí algoritmy předpokládaly, že máme MDP celé před sebou. Ve skutečnosti ovšem T ani R obvykle **neznáme**. Agent se proto musí učit ze zkušenosti, kterou získává interakcí s prostředím v diskrétních krocích, a tomuto přístupu se říká *model-free RL*.

Q-learning (Watkins, 1989)

Agent si udržuje *tabulku* $Q(s, a)$, tedy odhad diskontovaného součtu budoucích odměn při provedení akce a ve stavu s . Po každém přechodu $s \xrightarrow{a} s'$ s odměnou r přepíše příslušnou položku:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{cíl (TD target)}} - Q(s, a) \right], \quad 0 \leq \alpha \leq 1 \text{ je rychlost učení,}$$

což se dá zapsat i jako $Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a'))$.

Off-policy: cíl obsahuje $\max_{a'}$, takže se algoritmus učí hodnotu *optimální* politiky bez ohledu na to, jakou politikou agent skutečně jedná. Za předpokladu, že každá dvojice (s, a) je navštívena nekonečněkrát a že α vhodně klesá, konverguje $Q \rightarrow Q^*$.

Dilema průzkumu a využití; ε -greedy

Agent stojí před dvěma protichůdnými požadavky. Má *využívat* (*exploit*) to, co už umí, protože jinak zbytečně přichází o odměnu, ale zároveň musí *zkoušet* (*explore*) nové akce, protože jinak se o ničem lepším nedozví. Standardním řešením tohoto dilematu je ε -greedy politika:

$$\pi(s) = \begin{cases} \text{náhodná akce z } A & \text{s pravděpodobností } \varepsilon, \\ \arg \max_a Q(s, a) & \text{s pravděpodobností } 1 - \varepsilon. \end{cases}$$

Hodnota ε se typicky v čase snižuje (*annealing*). Alternativou je softmax neboli Boltzmannovo rozdělení $\pi(a|s) \propto e^{Q(s,a)/\tau}$, dále optimistická inicializace Q a UCB, tedy horní meze spolehli-

vosti.

Konkrétní krok Q-learningu. Necht' $\alpha = 0,5$ a $\gamma = 0,9$. Agent je ve stavu s_1 , provede akci a_1 , dostane odměnu $r = 10$ a přejde do s_2 . Aktuální hodnoty jsou $Q(s_1, a_1) = 4$, $Q(s_2, a_1) = 6$ a $Q(s_2, a_2) = 8$. Výpočet pak proběhne takto:

$$\begin{aligned}\max_{a'} Q(s_2, a') &= \max(6, 8) = 8, \\ \text{TD cíl} &= r + \gamma \max_{a'} Q(s_2, a') = 10 + 0,9 \cdot 8 = 17,2, \\ \text{TD chyba} &= 17,2 - Q(s_1, a_1) = 17,2 - 4 = 13,2, \\ Q(s_1, a_1) &\leftarrow 4 + 0,5 \cdot 13,2 = \mathbf{10,6}.\end{aligned}$$

SARSA

Název je zkratka za *State–Action–Reward–State–Action* a vyjmenovává přesně to, co aktualizace potřebuje. Na rozdíl od Q-learningu totiž použije *skutečně zvolenou* následující akci a' , tedy akci vybranou touž politikou, kterou agent jedná:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)].$$

On-policy: učí se hodnotu politiky, kterou agent skutečně provádí, včetně jejího náhodného průzkumu. Důsledkem je, že SARSA je *opatrnější*. V úloze „chůze po útesu“ (*cliff walking*) se Q-learning naučí optimální cestu těsně u propasti a při ε -greedy do ní občas spadne, zatímco SARSA se naučí bezpečnější cestu dál od okraje, protože započítá riziko vlastního průzkumu. Obě metody zobecňuje TD(λ) s *eligibility traces*, které rozprostírají kredit na více předchozích kroků.

6.4 Metody založené na politice

Q-learning i SARSA hledají politiku oklikou přes hodnotovou funkci. Druhá velká rodina metod tuto okliku vynechává.

Policy gradient a actor–critic

Namísto hodnotových funkcí lze optimalizovat *přímo politiku*. Politiku parametrizujeme vektorem z , tedy $\pi(a \mid s, z)$, a hledáme z^* gradientním výstupem po očekávaném výnosu. **REINFORCE** (Williams, 1992) odhaduje gradient z Monte Carlo simulací celých epizod:

$$z_{t+1} = z_t + \alpha G_t \nabla_z \ln \pi(A_t \mid S_t, z_t),$$

kde G_t je výnos od času t . Intuitivně předpis říká: zvyš pravděpodobnost akcí, po nichž následoval vysoký výnos. Proti hodnotovým metodám má dvě výhody, totiž že zvládá *spojité* akční prostory a přirozeně vytváří *stochastické* politiky, které jsou nutné v hrách se smíšenými ekvilibrii. Nevýhodou je vysoký rozptyl odhadu gradientu.

Actor–critic oba přístupy kombinuje: herec (*actor*) reprezentuje politiku a kritik (*critic*) se učí hodnotovou funkci a snižuje rozptyl. Přidá-li se **funkce výhody** $A(s, a) = Q(s, a) - V(s)$, škáluje se gradient relativní výhodnosti akce místo celého výnosu. Do rodiny patří varianty A3C, A2C, PPO, SAC a DDPG.

Hluboké RL: DQN nahrazuje tabulku Q neuronovou sítí. Klíčové jsou přitom dva triky: *experience replay* rozbíjí korelace mezi vzorky a *target network* stabilizuje cíl.

Proč hluboké RL potřebuje ty triky: smrtící trojice. Konvergenční záruky Q-learningu platí pro *tabulkovou* reprezentaci. Jakmile se sejdou tři věci najednou, může učení *divergovat*: **aproximace hodnotové funkce**, kdy tabulku nahradí neuronová síť, **bootstrapping**, kdy cíl aktualizace

obsahuje vlastní odhad $\max_{a'} Q(s', a')$, a **off-policy učení**, kdy data pocházejí z jiné politiky, než kterou se učíme. Sutton a Barto tuto trojici nazývají *deadly triad*. Každá dvojice sama o sobě je zvládnutelná; DQN se s trojicí vyrovnává právě replay bufferem a zamrzlou cílovou sítí, které oslabují druhý a třetí člen.

Tvarování odměny (*reward shaping*)

Je-li odměna řídká a agent dostane signál až v cíli, stává se učení prakticky neproveditelným. Odměna se proto doplňuje o průběžné nápovědy $F(s, a, s')$. Naivní doplnění ale mění optimální politiku: agent se naučí sbírat nápovědy místo řešit úlohu.

Ng, Harada, Russell (1999) ukázali, že doplnění tvaru

$$F(s, a, s') = \gamma \Phi(s') - \Phi(s)$$

zachovává množinu optimálních politik pro libovolnou *potenciálovou funkci* $\Phi : S \rightarrow \mathbb{R}$, a že je to jediný tvar, který to zaručuje pro všechny MDP. Intuice za tím je prostá: součet takových přírůstků po libovolné cestě teleskopicky závisí jen na koncových bodech, nikoli na tom, kudy agent šel, takže se cykly nevyplátí.

6.5 Multiagentní zpětnovazební učení

Dosud se učil jediný agent v prostředí, které se pod ním nemění. Jakmile se učí víc agentů naráz, přestává to platit a formalismus se musí rozšířit.

Markovská hra (*Markov game*, stochastická hra)

Markovská hra zobecňuje MDP na N agentů a má tvar $\langle N, S, (A_i)_{i \in N}, T, (R_i)_{i \in N}, \gamma \rangle$. **Sdružený akční prostor** je součin individuálních, $\mathbf{A} = A_1 \times \dots \times A_N$. Přejít závisí na akcích *všech* agentů a každý agent má *vlastní* odměnu R_i i vlastní politiku π_i , ale jeho hodnotová funkce závisí na politikách všech ostatních.

Spojení s teorií her je bezprostřední: politika agenta je *nejlepší odpovědí* na sdružené politiky ostatních, sdružené politiky mohou být v **Nashově ekvilibriu** a mohou být **Pareto-optimální**. Zajímavé jsou speciální případy. Pro $N = 1$ dostaneme MDP, hra s jediným stavem dává hru v normální formě a $N = 2$ s nulovým součtem dává *minimax* hru.

Minimax-Q (Littman, 1994) řeší hry dvou agentů s nulovým součtem. Místo $\max_{a'} Q(s', a')$ se použije hodnota smíšeného minimaxového ekvilibria matice $Q(s', \cdot, \cdot)$, což znamená v každém kroku vyřešit lineární program. **Nash-Q** dělá totéž pro obecný součet, konverguje ale jen za silných předpokladů.

Proč je MARL těžké

Potíže jsou čtyři a objevují se prakticky vždy:

- Agenti aktualizují politiky **současně**, takže z pohledu jednoho z nich je prostředí **nestacionární**. Tím se porušuje základní předpoklad konvergence RL a agent míří na „pohyblivý cíl“.
- Sdružený akční prostor roste **exponenciálně** s počtem agentů.
- Prostředí je jen **částečně pozorovatelné**, což vede na Dec-POMDP, jehož optimální řešení je NEXP-úplné.
- Objevuje se problém **přiřazení zásluh**: při společné odměně není jasné, který agent k ní přispěl.

Standardní protiopatření je *centralizované učení s decentralizovaným prováděním* (CTDE): kritik během tréninku vidí stav i akce všech agentů, zatímco herec při běhu vystačí jen se svými lokálními pozorováními (MADDPG, COMA, QMIX). Dále se používá sdílení parametrů mezi homogenními agenty a *opponent modelling*.

Self-play. Zvláštní postavení má situace, kdy se agent učí hraním proti sobě samému, případně

proti svým dřívějším verzím. V hrách s nulovým součtem a úplnou informací vede takový postup spolehlivě k silným strategiím, jak dokládají TD-Gammon, AlphaGo Zero a AlphaZero. Rizikem jsou *cyklické* preference strategií (kámen–nůžky–papír), u nichž naivní self-play osciluje, místo aby se zlepšoval; proto se zavádějí ligy protivníků a populační metody.

6.6 Shrnutí ke zkoušce

- Metody učení agentů se podle učebního signálu dělí na učení s učitelem, učení bez učitele a učení zpětnovazební. Vedle tohoto dělení stojí imitační učení spolu s inverzním RL a evoluční metody (LCS, neuroevoluce); dále se metody třídí podle toho, co se agent učí, tedy zda model prostředí, model protivníka, nebo přímo politiku, a podle toho, jestli se učí individuálně, nebo sociálně. Nejběžnějším přístupem je RL.
- **MDP** je pětice $\langle S, A, T, R, \gamma \rangle$ s markovskou vlastností; nad ní se definuje politika π a hodnotové funkce V^π a Q^π . U zkoušky je potřeba napsat Bellmanovu rovnici optimality a vysvětlit value iteration, která konverguje, protože Bellmanův operátor je kontrakce, i policy iteration.
- **Q-learning** aktualizuje $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$, je off-policy a tabulkový; jeden krok je potřeba umět spočítat s čísly.
- **SARSA** používá aktualizaci $\dots + \alpha[r + \gamma Q(s', a') - Q(s, a)]$, je on-policy a chová se opatrněji, což ukazuje úloha cliff walking. Dilema průzkumu a využití řeší ϵ -greedy politika.
- **Policy gradient** optimalizuje politiku přímo (REINFORCE, tedy $\nabla \ln \pi$ krát výnos) a **actor-critic** k němu přidává kritiku: herec je politika, kritik hodnotová funkce a výhoda je $A = Q - V$. A3C je asynchronní varianta, DQN přináší replay buffer a target network.
- **Smrtící trojice** aproximace, bootstrappingu a off-policy učení může vést k divergenci, a právě proto potřebuje DQN své triky. U **tvarování odměny** platí, že optimální politiku zaručeně nemění jen potenciálový tvar $F = \gamma \Phi(s') - \Phi(s)$.
- **Markovská hra** je MDP pro N agentů se sdruženým akčním prostorem a individuálními odměnami; ptáme se u ní na nejlepší odpověď, Nashovo ekvilibrium a Pareto-optimalitu. **Minimax-Q** řeší hry s nulovým součtem a v každé aktualizaci počítá LP, **Nash-Q** míří na obecný součet.
- MARL komplikují čtyři věci: nestacionarita prostředí, exponenciální sdružený akční prostor, částečná pozorovatelnost a přiřazení zásluh. Standardní odpovědí na ně je CTDE. Za zmínku stojí ještě self-play a AlphaZero.

7 Metodologie návrhu, jazyky a prostředí MAS

7.1 Agentově orientované softwarové inženýrství

Objektově orientované metodologie a jejich notace UML na multiagentní systém nestačí, a to hned ze tří důvodů. Agent je autonomní, takže mu nelze přímo zavolat metodu a očekávat, že ji provede. Agent má vlastní cíle, které se nemusejí krýt s cíli toho, kdo ho volá. A vztahy mezi agenty nejsou strukturální, nýbrž *organizační*: popisují se rolemi, protokoly a závazky, ne dědičností a agregací. Právě proto vznikly samostatné metodologie **AOSE** (*agent-oriented software engineering*). Popisují systém v pojmech rolí a interakcí mezi nimi, a ne v pojmech tříd a volání metod.

Gaia (Wooldridge, Jennings, Kinny, 2000)

Jedna z nejcitovanějších AOSE metodologií. Její klíčovou metaforou je **organizace**: multiagentní systém se popisuje tak, jako by šlo o organizaci, v níž agenti hrají **role**. Gaia postupuje od požadavků přes analýzu k návrhu, samotnou implementaci nechává na konkrétní platformě. Analýza a návrh přitom tvoří dvě oddělené vrstvy popisu: nejprve se systém zachytí abstraktně, v rolích a jejich interakcích, a teprve pak se tento popis převede na konkrétní typy agentů a jimi poskytované služby.

Fáze analýzy staví dva abstraktní modely. *Model rolí* (*roles model*) popisuje každou roli čtyřmi atributy. *Permissions* určují, jaké zdroje smí role číst a měnit. *Responsibilities* říkají, co má role zajistit, a dělí se na dvě skupiny: *liveness* vlastnosti znějí „něco dobrého se stane“ a popisují

se regulárním výrazem nad aktivitami a protokoly, kdežto *safety* vlastnosti znějí „nic špatného se nestane“ a popisují se invarianty. *Activities* jsou výpočty, které role provádí sama, zatímco *protocols* zachycují její interakce s jinými rolemi. Druhý model, *model interakcí*, tyto protokoly mezi rolemi definuje: jejich účel, iniciátora, respondéra, vstupy a výstupy.

Fáze návrhu z obou abstraktních modelů odvozuje tři konkrétní. *Model agentů* přiřazuje role typům agentů a stanoví počty instancí. *Model služeb* popisuje funkce, které role poskytuje, tedy jejich vstupy, výstupy, předpoklady a efekty. *Model známostí* (*acquaintance model*) je orientovaný graf zachycující, kdo s kým komunikuje; slouží ke kontrole úzkých hrdel a přílišné provázanosti.

Omezení: původní Gaia počítá s uzavřeným systémem, kooperativními agenty a statickou organizací. Rozšíření *ROADMAP* a *Gaia v.2* tato omezení uvolňují.

♦ nad rámec slidů: slidy jmenují jen *Gaiu* a *role*; ostatní metodologie jsou orientační] Gaia zdaleka není jediná. **Prometheus** (Padgham, Winikoff, 2002) je praktická metodologie mířená na **BDI agenty** a má tři fáze. Ve *specifikaci systému* se stanoví cíle, *funkcionality* a rozhraní s okolím, tedy *percepts* a *actions*. *Architektonický návrh* pak určí typy agentů, protokoly a zprávy. *Detailní návrh* nakonec sestoupí ke capabilities, plánům a událostem, což jsou přímo stavební kameny BDI.

Tropos staví na modelování *aktérů a cílů* pomocí frameworku *i** a jako jediný pokrývá i fázi raných požadavků. Pracuje s pojmy *actor*, *goal* a *softgoal*, což je nefunkční požadavek bez ostrého kritéria, a dále s pojmy *plan*, *resource* a *dependency*. Vedle těchto tří se používají ještě **MaSE**, **INGENIAS**, **ADELFE** a **PASSI**.

7.2 Standard FIPA a architektura platformy

Metodologie říká, jak systém navrhnout. Standard FIPA odpovídá na jinou otázku: jak má vypadat prostředí, ve kterém navržený agenti poběží a budou spolu mluvit. Popisuje proto referenční architekturu platformy, tedy komponenty, ze kterých se agentní platforma skládá, a úlohy, jež každá z nich plní.

Referenční architektura agentní platformy podle FIPA

- **AMS** (*Agent Management System*) je povinná komponenta a funguje jako „matrice“ platformy: eviduje agenty, přiděluje jim jednoznačné *AID* (agent identifier), řídí jejich životní cyklus (create, suspend, resume, terminate) a poskytuje bílé stránky (*white pages*).
- **DF** (*Directory Facilitator*) hraje roli žlutých stránek (*yellow pages*). Agenti u něj registrují služby, které nabízejí, a naopak v něm vyhledávají poskytovatele služeb.
- **MTS** (*Message Transport Service*) doručuje ACL zprávy jak uvnitř jedné platformy, tak mezi platformami; využívá k tomu protokoly IIOP a HTTP a kódování bitefficient, XML nebo string.
- **ACC** (*Agent Communication Channel*) tvoří rozhraní k MTS.

Standard nekončí u architektury: definuje dále jazyk ACL, obsahové jazyky (SL), interakční protokoly a správu ontologií.

7.3 Jazyky a prostředí

♦ nad rámec slidů: **AGENT-0** a **Concurrent MetateM** ve slidech nejsou; **JADE** a **Jason** ano] Historickým východiskem je **agentově orientované programování** (Shoham 1993). V tomto paradigmatu tvoří stav programu *mentální kategorie*, tedy přesvědčení, závazky a schopnosti, a program sám je množinou *pravidel závazků*. Prvním jazykem tohoto druhu byl **AGENT-0**. Jinou cestou jde **Concurrent MetateM**, kde se specifikace a program nerozlišují: agent je *přímo spustitelnou specifikací* v temporální logice. Oba jazyky navazují na linii deduktivních agentů (odst. 2.1). Prakticky nejúspěšnějším agentním jazykem je dnes AgentSpeak/Jason a právě jemu spolu s dalšími třemi systémy se věnuje následující přehled.

Systém	Typ	Charakteristika
JADE	Java, FIPA platforma	<i>Java Agent DEvelopment Framework</i> , jedna z nejrozšířenějších platform pro MAS, plně v souladu s FIPA. Agent = třída dědící z Agent s metodou <code>setup()</code> ; chování se skládá z objektů Behaviour (OneShot , Cyclic , Ticker , FSMBehaviour , SequentialBehaviour), které plánuje kooperativní (nepreemptivní) plánovač uvnitř jednoho vlákna agenta. Zprávy: ACLMessage , <code>send()/blockingReceive()</code> , MessageTemplate . Hotové třídy pro protokoly (ContractNetInitiator , AchieveREResponder), ladicí nástroje (RMA, Sniffer, Introspector), kontejnery a mobilita agentů.
Jason	interpret Agent-Speak(L) v Javě	Referenční implementace AgentSpeak , tedy BDI jazyka (logické programování + BDI), přímý potomek PRS. Program agenta = počáteční <i>beliefs</i> , <i>rules</i> a <i>knihovna plánů</i> ve tvaru <code>spouštěč : kontext <- tělo</code> . Spouštěčem je <i>událost</i> (+b přidání přesvědčení, +!g přidání cíle, -!g selhání cíle). Cyklus interpretu: vnímej → aktualizuj beliefs → vyber událost → najdi <i>relevantní</i> plány (podle spouštěče) → vyber <i>aplikovatelné</i> (podle kontextu) → vytvoř záměr → vykoněj krok. S CArtAg0 (prostředí) a Moise (organizace) tvoří <i>JaCaMo</i> .
NetLogo	prostředí pro ABM	Jazyk odvozený od Logo pro <i>agent-based modeling</i> . Entity: <i>turtles</i> (mobilní agenti), <i>patches</i> (buňky prostředí), <i>links</i> a <i>observer</i> . Knihovna modelů (Segregation, Wolf Sheep Predation, Ants, Flocking), <i>BehaviorSpace</i> pro parametrické studie. Určeno pro emergentní chování a výuku, ne pro FIPA komunikaci.
SPADE	Python, XMPP	<i>Smart Python Agent Development Environment</i> . Komunikace přes standardní XMPP místo vlastního MTS, agent = třída s <i>behaviours</i> (CyclicBehaviour , PeriodicBehaviour , FSMBehaviour), asyncio, webové rozhraní; populární díky napojení na Python ekosystém strojového učení.

Další prostředí. Vedle čtyř systémů z tabulky stojí za zmínku tři skupiny nástrojů. *Jadex*, *JACK* a *JAM* jsou další implementace BDI/PRS, přičemž *Jadex* staví BDI vrstvu nad JADE. *MASON*, *Repast* a *GAMA* slouží jako simulační platformy pro agentní modely. *PettingZoo*, *Gymnasium* a *OpenSpiel* pak nabízejí standardní prostředí pro multiagentní zpětnovazební učení.

7.4 Agenti postavení na velkých jazykových modelech

♦ nad rámec slidů: nově ve slidech verze 2025 (kapitola *LLM Agents*)

Poslední přírůstek do rodiny mechanismů výběru akcí (§1.9) nepoužívá ani dedukci, ani pravidla, ani naučenou Q -funkci. Rozhoduje za něj **velký jazykový model** (LLM). Agent se pak scvrkne na smyčku, v níž se stav prostředí zakóduje do textového promptu a výstup modelu se interpretuje jako akce. Rozhodovací logika tedy nesídlí v kódu agenta, nýbrž v modelu; kód kolem něj jen sestavuje prompt a překládá odpověď zpět na akci.

LLM v roli komponenty agenta

- **LLM jako politika** (*action selection*). Prompt obsahuje popis role, stav prostředí a seznam dostupných akcí a výstupem je zvolená akce. Model tak plní funkci $action : I \rightarrow A$ z abstraktní architektury, přičemž vnitřním stavem I je kontextové okno, případně externí paměť.
- **LLM jako plánovač**. Model rozloží cíl na podcíle a generuje kandidátní plány. Prohledávání prostoru plánů řídí buď model sám (*chain of thought*, *ReAct*), nebo vnější prohledávací algoritmus, jemuž model slouží jako generátor nástupníků a zároveň jako heuristika.
- **LLM jako kritik**. Ve schématu *actor-critic* (§6.4) model hodnotí kvalitu provedené akce nebo celé trajektorie, a poskytuje tak zpětnou vazbu tam, kde prostředí skalární odměnu

nedává.

- **LLM jako rozhraní k nástrojům** (*tool use*). Model volí, který externí nástroj zavolat a s jakými argumenty; tím nástrojem bývá vyhledávání, kalkulač, API nebo jiný agent. Je to přímá obdoba *služeb* registrovaných v DF a jejich volání přes ACL.
- **Učení s člověkem ve smyčce** (*human-in-the-loop*). Člověk opravuje a doplňuje zpětnou vazbu a podle ní se upravuje prompt, paměť nebo přímo váhy modelu.

Typické nasazení: agentové simulace. Nejčastější aplikací nejsou optimalizační úlohy, nýbrž *agent-based modeling* (§7.3, NetLogo). LLM agent tam nahradí jednoduchého reaktivního agenta a dodá mu doménovou znalost i jazykové chování:

- *Šíření informací v sociální síti*. Agenti jsou uzly grafu sociálních interakcí a zprávu si předávají v přirozeném jazyce; sleduje se, jak rychle a jakým tvarem se zpráva sítí šíří.
- *Syntetický volební panel*. Populace agentů se vygeneruje podle sociologických průzkumů, tedy podle demografie a postojů, a použije se k odhadu výsledků voleb nebo reakce na kampaň.

Zařazení a kritika. Z hlediska okruhu jde o další rodinu mechanismů výběru akcí, subsymbolickou stejně jako neuronové sítě, ale s výrazně širší doménovou znalostí a bez nutnosti trénovat model úlohy. Slabiny jsou zrcadlovým obrazem těchto předností. Agent nemá explicitní model světa, takže o něm nelze dokazovat tvrzení jako u deduktivního agenta. Sémantika jeho „přesvědčení“ je **neverifikovatelná** ve stejném smyslu jako u FIPA-ACL (§4.7). Chování je nedeterministické a cena jednoho rozhodnutí je o několik řádů vyšší než u pravidlového agenta.

7.5 Shrnutí ke zkoušce

- Objektově orientované metodologie nestačí proto, že agent je autonomní a nelze mu jen zavolat metodu, že má vlastní cíle a že vztahy mezi agenty jsou organizační, nikoli strukturální.
- **Gaia** popisuje systém jako organizaci složenou z rolí. Analýza dává model rolí (permissions, responsibilities s liveness a safety vlastnostmi, activities, protocols) a model interakcí, návrh pak model agentů, model služeb a model znalostí. Metodologie předpokládá kooperativní agenty a statickou organizaci.
- **Prometheus** cílí na BDI a postupuje od funkcionalit → přes typy agentů → ke capabilities a plánům, kdežto **Tropos** pracuje s aktéry, cíli a softgoals a se závislostmi mezi nimi a jako jediný pokrývá i rané požadavky.
- **FIPA platforma** stojí na třech komponentách. AMS spravuje bílé stránky, životní cyklus agentů a přidělování AID, DF plní úlohu žlutých stránek s nabízenými službami a MTS spolu s rozhraním ACC doručuje ACL zprávy uvnitř platformy i mezi platformami.
- **JADE** je javová FIPA platforma stavěná z tříd **Agent**, **Behaviour** a **ACLMessage**, s hotovými protokoly a ladicími nástroji. **Jason/AgentSpeak** je BDI jazyk s plány tvaru **spouštěč : kontext** <- **tělo** a potomek PRS. **NetLogo** slouží k ABM simulacím s entitami turtles, patches a links a **SPADE** kombinuje Python s XMPP.
- **LLM agenti** používají jazykový model jako *politiku* (prompt → akce), jako *plánovač*, jako *kritika* ve schématu actor-critic a jako rozhraní pro *volání nástrojů*. Nasazují se hlavně v agentových simulacích, ať už jde o šíření zpráv sítí, nebo o syntetický volební panel. Tvoří další rodinu mechanismů výběru akcí: nemají explicitní model světa a sémantika jejich přesvědčení je neverifikovatelná, zato disponují širokou doménovou znalostí.

Závěrem: jak na zkoušku

Okruh je široký a zkoušející obvykle ověřuje, nakolik uchazeč rozumí *souvislostem*. Otázky se točí kolem čtyř os:

1. **Reaktivita vs. proaktivita.** Tato osa vede od definice inteligentního agenta přes strategie odhodlání (bold vs. cautious) a parametry γ/ϕ v síti Maes až k vrstvám hybridních architektur.

2. **Individuální vs. kolektivní racionalita.** Sobecký agent dovede systém k (D, D) ve věžňově dilematu, k tragédii obecní pastviny i k taktickému hlasování. Odtud plyne potřeba mechanism designu (Vickrey, VCG), tedy návrhu *pravidel*, za nichž se i sobeckému agentovi vyplatí být pravdomluvný.
3. **Symbolická vs. subsymbolická reprezentace.** Na jedné straně stojí FOL, STRIPS, ontologie a ACL, na druhé subsumpce, aktivační sítě a neuronové sítě v RL. Praxe je hybridní.
4. **Teorie vs. praxe.** Proti elegantním, ale nekonstruktivním definicím (optimální agent, Nashova věta, Arrowova věta) stojí použitelné algoritmy (A^* , CBS, Q-learning, CNET, JADE). K tomu patří i složitost: hledání NE je PPAD-úplné a WDP i optimální MAPF jsou NP-těžké.

Zbývá výčet drobností, na které se ptají nejčastěji: rozdíl mezi *desire* a *intention*; proč efektem *inform* není, že příjemce uvěří; proč je (D, D) jediné *ne*-Pareto-optimální řešení věžňova dilematu; proč je pravdomluvnost dominantní ve Vickreyově aukci; jaký je rozdíl mezi off-policy Q-learningem a on-policy SARSA; a co dělá CBS na vysoké a co na nízké úrovni.